Department of Computer Science

Submitted in part fulfilment for the degree of MSc in Cyber Security

Implementing an Intrusion Detection and Prevention System using Software-Defined Networking: Defending against SYN Flood, Port Scanning and ICMP Flood Attacks- a Controller Comparison

Emmanuel Orji

09 September 2024

Number of pages = 47

Supervisor: Vasileios Vasilakis

ACKNOWLEDGEMENTS

This project is dedicated to God, and my mother.
I will like to thank my family and friends for their immense support and encouragement.
I also wish to express my gratitude to my supervisor, Dr Vasileios Vasilakis for his guidance and support.

STATEMENT OF ETHICS

The author, Emmanuel Orji, declares that this project titled Implementing an Intrusion Detection System using Software-Defined Networking and any work presented within it are their own. They confirm that:

- The project was completed exclusively or chiefly whilst a MSc Cyber Security candidate at the University of York.
- All source materials used have been acknowledged, whether they are directly quoted or not.
- The University's Academic Integrity Misconduct Policy, Guidelines and Procedures - https://www.york.ac.uk/staff/supporting-students/academic/taught/misconduct/ have been adhered during the project.
- Any assistance with proofreading or editing has been acknowledged and taken place in accordance with the University's Guidance on Proofreading and Editing - https://www.york.ac.uk/media/abouttheuniversity/supportservices/academicregistry/registryservices/sca/v.%20Oct%202019_%20Guidance%20on%20proofreading.pdf
- No element of the project has been previously submitted for any qualification at the University of York or another academic institution.
- Programming code that has been generated from existing sources has been appropriately acknowledged.

TABLE OF CONTENTS

CONTENTS

TABLE OF FIGURES

TABLE OF TABLES

ABBREVIATIONS

ACK	Acknowledge
DoS	Denial of Service
ICMP	Internet Control Message Protocol
IDS	Intrusion Detection System
IDPS	Intrusion Detection and Prevention System
IP	Internet Protocol
MAC	Media Access Control
OF	Open Flow
OS	Operating System
OvS	Open vSwitch
OVSDB	Open vSwitch Database Management
RL	Rate Limiting
RTT	Round-Trip Time
SDN	Software Defined Networking
SEK	Security Enforcement Kernel
SYN	Synchronize
TCP	Transport Control Protocol
TRW	Credit-Based Threshold Random Walk
VM	Virtual Machine

Executive summary

This paper explores the implementation of an Intrusion Detection and Prevention System (IDPS) using Software-Defined Networking (SDN) to defend against common Denial-of-Service (DoS) attacks, specifically SYN Flood, Port Scanning, and ICMP Flood attacks. The study focuses on evaluating three widely-used SDN controllers—POX, Ryu, and Faucet and comparing their performance in mitigating these threats.

The research is motivated by the escalating sophistication of cyberattacks, which demand robust and responsive network security solutions. SDN, with its decoupled control and data planes, offers a programmable framework that enhances the flexibility and scalability of security measures across a network.

A network testbed was created mostly using Ubuntu Virtual Machines (VM) and enlisting an hypervisor (Virtual Box), Open vSwitch (OvS) acting as the virtual switch, a VM that served as the SDN controller. Three IDPS scripts were used to detect and mitigate the attacks (one for each attack), two of which are bespoke scripts, whilst one was sourced from a top paper with slight changes made to it. The IDPS employed Credit- Based Threshold Random Walk and Rate limiting algorithms. Key performance metrics, including detection time, mitigation time, and the impact on legitimate traffic by measuring RTT, were recorded and analyzed.

The results indicate that while both controllers were capable of detecting and mitigating the attacks, the POX controller demonstrated superior performance in terms of speed of response and minimal impact on legitimate traffic. POX’s simplicity, speed and ease of integration with scripts made it a suitable choice for environments where quick threat detection and mitigation is critical. The Ryu controller, although slightly slower offers flexibility and scalability no less, and might be better suited for environments that require advanced customization and modularity with the obvious trade-off in threat attack mitigation speed.

The findings also suggest areas for future research, including the exploration of additional attack vectors and the integration of advanced techniques such as machine learning to enhance detection capabilities. Overall, this paper contributes valuable insights into the implementation of secure SDN-based networks, highlighting the potential of SDN controllers in improving network security.

1 Introduction

This chapter exists to help us understand why this project was conducted and why it is important. It documents the context, motivation, objectives as well as contributions of this project. Lastly, it highlights a general structure of this report.

1.1 Context

Recent years has seen the proliferation of networked systems. This is owed largely to cyber threats evolving quickly and reinforcing the necessity for robust network security. Significant development in areas such as Internet of Things (IoT), cloud computing has led to sophisticated cyber-attacks which challenges traditional IDS approaches [1]. The dynamic nature of these attacks prompts a less static approach that traditional IDS systems offer, and instead requires programmable and automated security configurations.

Denial of Service (DoS) attacks are still one of the highest occurring attacks launched by attackers to shut down hosts or networks [2]. This type of attack deliberately attempts to stop benign users from accessing particular network resources. Thankfully, software defined networking presents ways to detect and mitigate this existential threat.

Software defined networking characterized by its decoupled control and data planes offers a programmable and centralized framework that allows for comprehensive and cohesive security policies across the network [3]. It enhances flexibility, scalability and overall responsiveness in detecting and mitigating network security threats. With a global market of 24 billion U.S. dollars in revenue and a projection of 60 billion U.S dollars by 2028, we expect an increased adoption of this technology [4]. Another Study predicts significant SDN growth of 21.75% per year in data centres and within IoT domain [5].

The concern however remains its security, and this grows increasingly important with widespread adoption. This work aims to implement an Intrusion Detection and Prevention System (IDPS) using POX, Ryu, and Faucet controllers to detect and mitigate SYN Flood, Port scanning and HTTP Flood attacks (all forms of DoS attacks) and draw a comparison on which performs better in terms of speed of detection and prevention.

1.2 Motivation

With various SDN controllers representing diverse approaches to handling SDN environments, existing research has explored the general performance of SDN controllers [6] [7] [8], but overlooked how these controllers perform under the stress of active cyberattacks

hence lacking security-oriented comparisons. A security analysis into the speed of detection and mitigation of some of the most popular SDN controllers- like the ones used in this project, against attacks such as SYN Floods that accounted for 24% of all global network-layer attacks in 2021 [9], and port scanning which is frequently used in reconnaissance phase of cyberattacks, adds a crucial dimension to the evaluation of SDN controllers. This helps us gain insights which would be beneficial to network administrators and researchers, and also for future environments where they might be deployed.

1.3 Objectives

The objectives which this report will cover and attempt to accomplish, are based upon the previously discussed motivation and context around SDN networks. These are as follows:

1. To design and implement an Intrusion detection and prevention system against SYN flood attack, Port scanning attack and ICMP flood attack.
2. To integrate this IDPS with RYU controller, POX controller and Faucet Controller.
3. To test the IDPS and measure speed of detection and prevention against the attacks, on the controllers and draw a comparison.

1.4 Project Contribution

This project has developed an SDN-based Intrusion Detection and Prevention System (IDPS) designed to defend against SYN flood attack, Port scanning attack and ICMP flood attack and measure the detection and mitigation time for those attacks by each SDN controller.

Innovative IDPS scripts were written for both flood attacks, while minor changes were made to the script for the Port scanning attack, gotten from Birkinshaw et al., [10]. These scripts were catered to integrate with the controllers individually as they operate differently and use different libraries.

Testing the IDPS involved 3 experiments, and also measured its impact on legitimate traffic to accurately draw a comparison. From a security point of view, the POX controller emerged as the best option.

1.5 Structure

The subsequent parts of this thesis are organised in the following manner:

Chapter 2 gives background information needed to understand the SDN Paradigm and testbed environment.

Chapter 3 provides comprehensive review of related work and research in Intrusion detection and prevention within the SDN paradigm in relation to detecting and mitigate DoS attacks in SDN Controllers.

Chapter 4 focuses on the methodology behind IDPS design, and attacks

Chapter 5 talks about the testbed to be used, it's design and implementation

Chapter 6 describes the experiments conducted to test each controller, the results and finally draws a comparison.

Chapter 7 concludes the work done, providing a summary on comparison, checking if objectives were met. It also recognises what limitations exist allowing the definition of future work in the field.

2 Background

2.1 Traditional Computer Networks

Traditional Computer networks have been foundational to the development of digital communication as we know it. It utilizes individual networking devices such as routers or switches to send and receive packets between each other. However, their inherent complexities and limitations increasingly serve as obstacles to much desired innovation, efficiency and general management [3].

These routers and switches operate through three distinct logical planes: control, data forwarding and management. The control plane is responsible for drawing the network topology and maintaining the routing table, effectively acting as the device's "intelligence." The data plane handles the physical movement of packets from one network interface to another acting as instructed by the control plane, while the management plane oversees the configuration and monitoring of the network system as a whole [11].

2.1.1 Difficulties with this model

This approach to coupling the control plane to the data plane or “ossification” of computer networks as called my McKeown et al [12] has been satisfactory for a good number of years. However, network devices have become increasingly complex and in need for more independent and perhaps autonomous processing. Interestingly, network misconfigurations and related errors are extremely common in today’s networks. Feamster et al. alluded to more than 1000 configuration errors observed in BGP routers alone [13]

Also, vendors have come up with different sets of management standards and features making it difficult for interoperability between devices. Many would rather use these vendor proprietary hardware and systems than be innovative and develop open-source solutions which would allow for a more agile and adaptable networking environment. Programmability and great control over a large geographical networking environment, comprising a vast amount of network devices is desired and this is easily possible with Software Defined Networking.

2.2 Software Defined Networking

Software Defined Networking or SDN is an innovation in the field of programmable networks, with the goal of increasing the flexibility and agility of networks. The Open Network Foundation (ONF), a consortium devoted to the development, standardization and commercialization of SDN defined the technology as the physical separation of the network control plane from the forwarding plane [14]. In traditional networks, the control plane, data plane (or forwarding

plane) and management plane is housed by a single device. Therefore, each network device is coupled with all three.

However, in SDN, the control plane is separated from the data plane, allowing it to provide logic and make decisions while data plane just forwards traffic back and forth. The control and data planes communicate through a protocol called OpenFlow. It is one of the pillars of SDN and represents a standard method of transmitting forwarding rules from the controller (control plane) to the forwarding devices i.e. routers and switches [15]. This mechanism is how SDN controllers control the behaviour of the network and allow for a dynamic shaping of traffic flows. Figure 1 shows what an SDN network looks like in comparison to a traditional network.

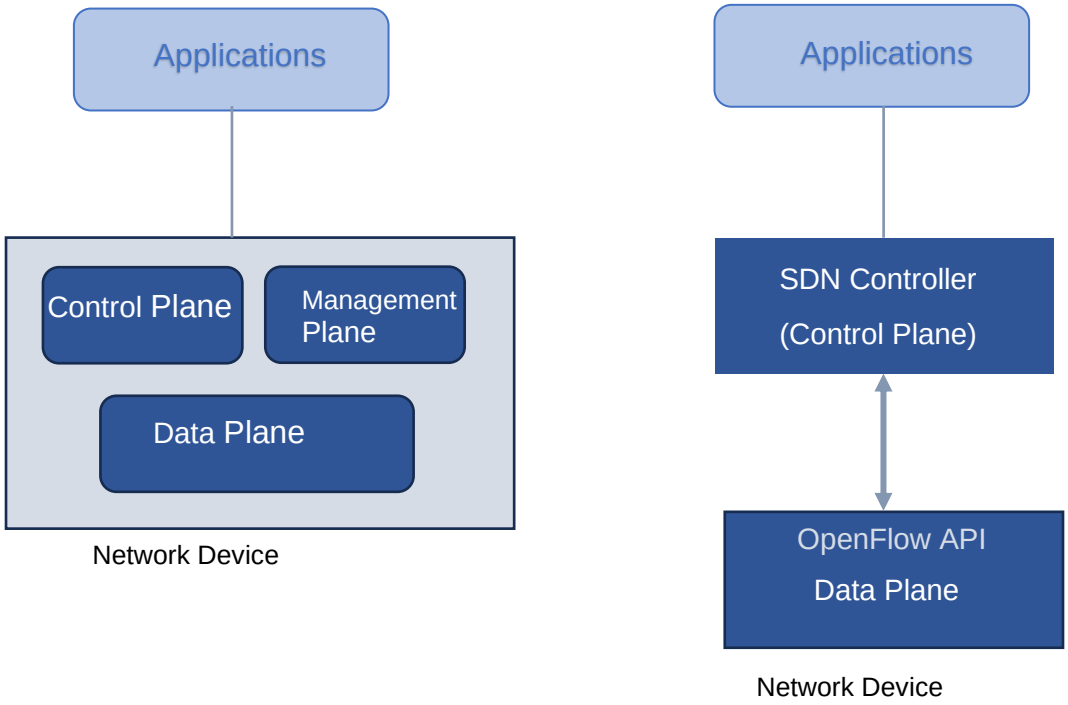

Figure 1: Traditional network (left) vs. SDN (right)

2.2.1 SDN Components

Here we will briefly discuss the key components within an SDN Network. This is according to Software defined networking comprehensive survey [3] and the Open networking Foundation [16]:

1. SDN Controller: This is the core and central component of the SDN Network. It acts as the brain of the network, controlling data flow from the applications down to the forwarding devices. It operates by utilizing the Northbound API to communicate with the applications and services and the Southbound API to communicate with the network devices. Examples include POX controller [17], Faucet, Ryu controller which would be looked at in greater detail later in this report.

2. Forwarding Devices (FD): These are the devices that make up the data plane of the SDN network. They can be physical or virtual and their function is to forward packets according to flow rules and instructions specified by the SDN Controller. They receive instruction via the Southbound API or interface.

3. Southbound API: This interface facilitates the communication between the SDN controller and forwarding devices. The instructions set for the forwarding devices is defined by the Southbound API. There are several protocols used for this like NETCONF, OVSDB, OpenFlow, with OpenFlow being the most common [3].

4. Applications: Network applications utilize the SDN controller's northbound APIs to implement various network services and functions.

5. Northbound API: They enable communication between the applications running, and the SDN controller, allowing the applications to request services from the network.

It is important to note that the SDN Controller stands as the control Plane, the forwarding devices belong to the data plane whilst the Applications or services belong to the Application plane. They are interconnected and communicate via their respective APIs or interfaces. Figure 2 below illustrates this relationship graphically.

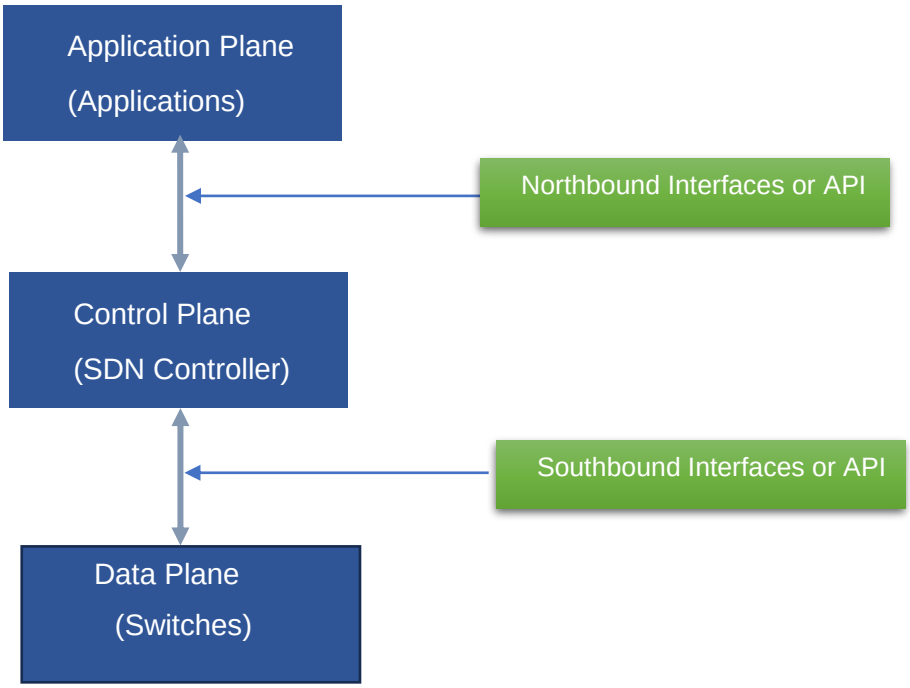

Figure 2:SDN Planes

2.3 OpenFlow

According to Nick Mckeown, OpenFlow provides an open protocol to program the flow table in different switches and routers [12]. Nick Mckeown, from Standford University led a team of researchers towards the development of new network protocols that could work on existing infrastructures. This draws significance from the fact that Organizations and are unable to readily experiment on their live infrastructures because new ideas can be disruptive and it is difficult to measure the size of impact it could have in their infrastructure. However, Universities are generally free from such operational constraint, hence the Mckeown research into OpenFlow.

We will now look at the main components of OpenFlow and how they work together.

2.3.1 OpenFlow Operation

According to Mckeown et. al. an OpenFlow architecture consists of an OpenFlow-complaint switch, a secure channel, and a controller. He outlined the fundamental operation of an OpenFlow solution as being a controller populating a switch's flow table with entries, then the

switch assesses incoming packet headers for a matching flow with an associated action. Where no matching flow is discovered, the switch then forwards the packet to the controller who provides instructions on how the packet should be processed. As the controller creates new flow entries, the switch is updated with it as well so it can evaluate related packets without having to contact the controller.

Also important to note the following about the communication between controller and switch:

- Every message between controller and switch begins with an OpenFlow header
- This header carries the OpenFlow version number, length, type and ID.
- The messages are divided into three categories which are the symmetric, asynchronous and controller-to-switch [18]
- The SDN controller fills the switch's forwarding table, ensuring flow tables are installed on it. This is because constantly requesting forwarding details from the SDN Controller is resource heavy and not as efficient.
- If the switch, however, requires forwarding instructions it asks the SDN controller.

2.3.2 Security between Controller and Switch in OpenFlow

One area of security in OpenFlow is the OpenFlow switch maintaining a secure channel with the controller. OpenFlow is placed above the TCP transport layer protocol, the switch-controller communication is configured to use Secure Socket Layer or Transport Layer Security ensuring that traffic is encrypted, although unencrypted traffic is allowed as well.

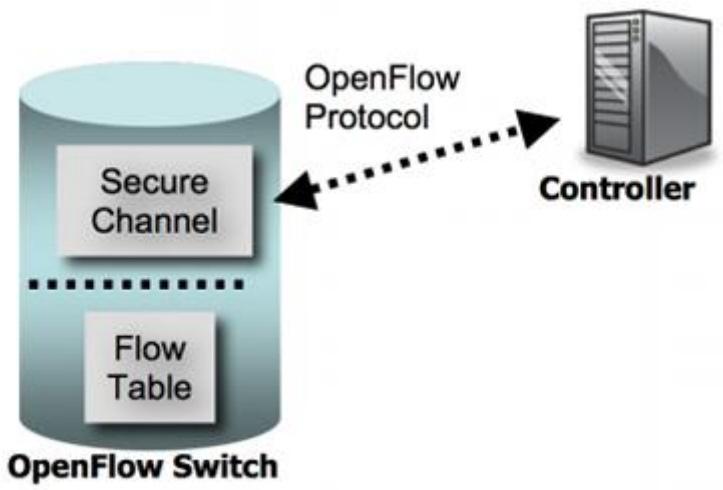

Figure 3: OpenFlow Switch Components *[16]*

There are two types of packets generally known to both the controller and switch, they may encapsulate other types of OpenFlow packets. These are:

1. **OFPT_PACKET_IN**: As earlier mentioned, the switch is unable to process some packets so it sends them to the controller, and that's what these packets are. This typically happens because a flow-entry for the packet does not exist. Every SDN switch has a flow-entry called table-miss which defines instructions for a switch to do where there is no matching flow-entry.

2. **OFPT_PACKET_OUT**: these are packets sent as a response to an OFPT_PACKET_IN from the controller to a switch. OFPT_FLOW_MOD packets carrying flow-entry information can be encapsulated within OFPT_PACKET_OUT packets as well.

2.3.3 Flow table & Matching Algorithm

An OpenFlow switch has one or multiple flow tables. These flow tables contain sets of flow entries matching packets and the associated action to take on them. These flows are assigned from the SDN controller. The following components make up a flow table entry:

i. Match field: They match packets to one or several of the following- ingress port, packet headers, metadata from previous table [16]
ii. Priority: They dictate the flow entry precedence
iii. Counters: They update statistics for a flow such as matched packets, number of bytes, duration of the flow
iv. Instructions: They specify how to handle matching packets based on an action set
v. Timeouts: This indicates the longest period of time that can pass before the switch ends the flow.
vi. Cookie: Some controllers use this to filter flow statistics, modifications or deleted flows.

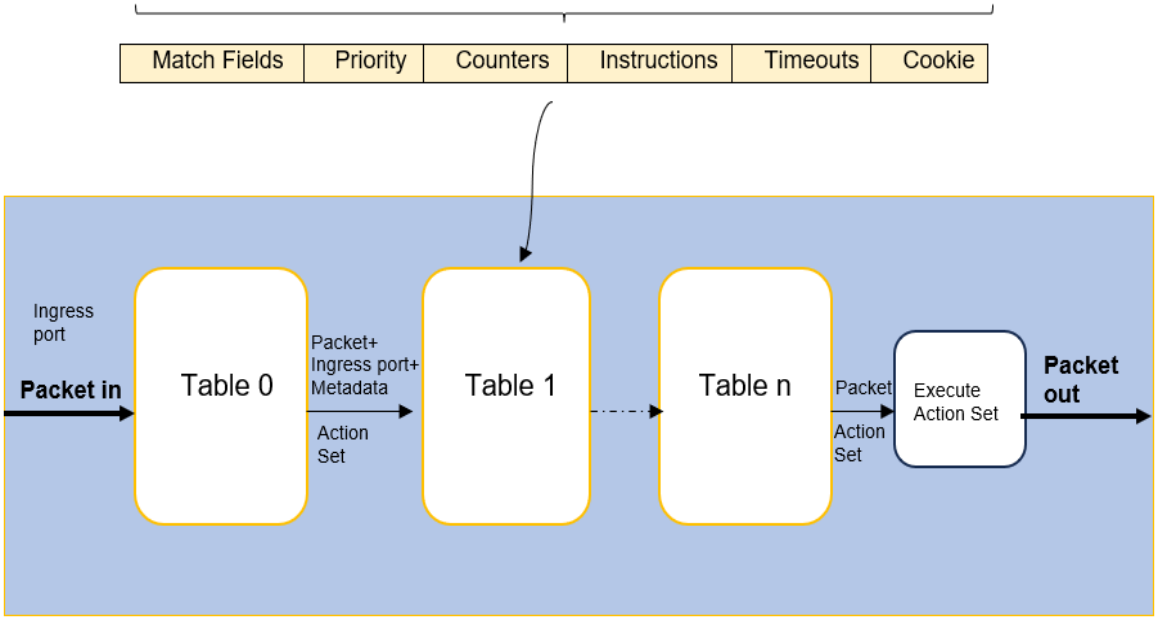

Figure 4: Flow tables in an SDN architecture

While incoming packets are processed based on entries in the flow table, this is mostly done through a matching algorithm. Packet match fields are extracted from the incoming packet. The switch performs a lookup of header fields such as IP destination address or Ethernet source address, ingress port or metadata fields (used to pass information between tables in a switch) [16]. After this, a lookup is done to see if it matches any flow entry or if any of the fields used for the lookup matches those defined in the flow table entry. Where any flow table entry field has an omitted value, it tries to match the packet with an entry that has all possible values in the packet header.

It is crucial to note that the SDN controller sets a priority to each flow entry in the flow table. Wildcarded parameters in a flow entry are assigned a priority and high priority parameters are used to match packets before low priority parameters are used. Where duplicate matching flow entries for a packet have the same priority, the switch on authority selects one of the best matching flow entries

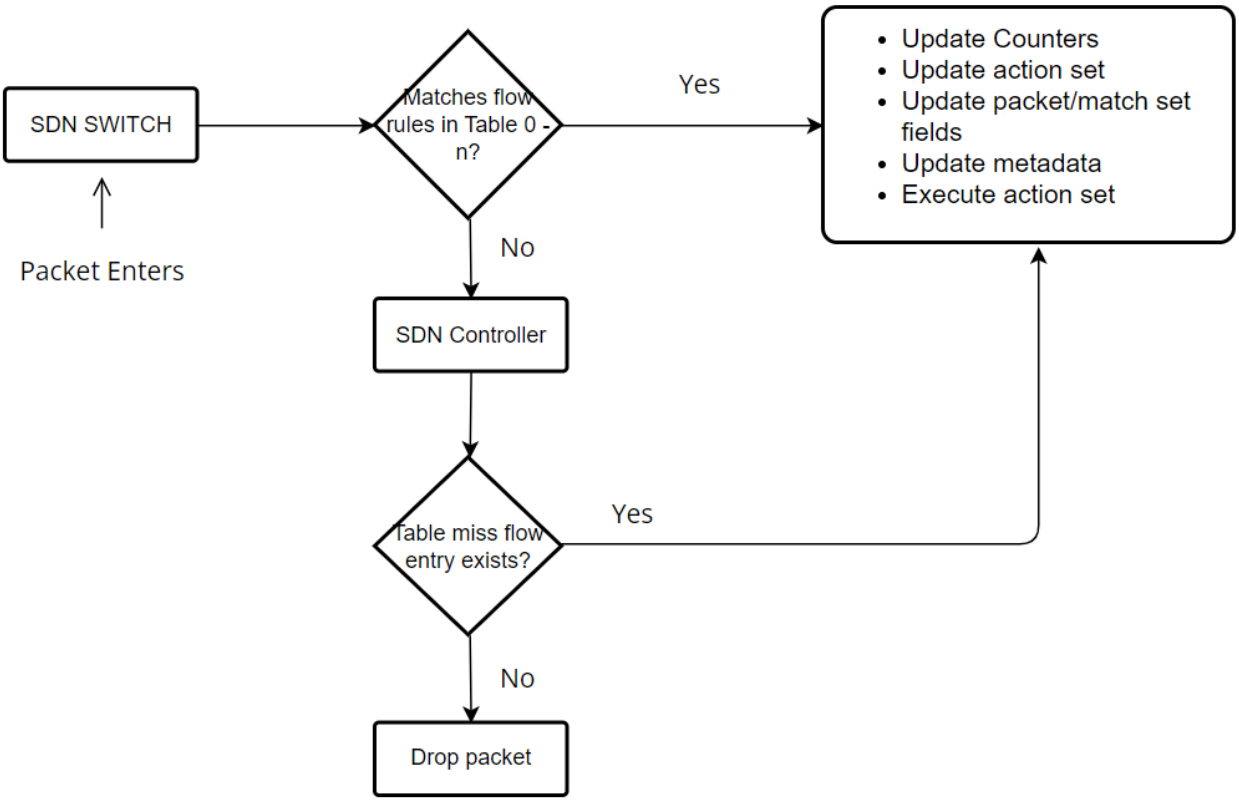

Figure 5: Packet matching flowchart

2.4 Open vSwitch

The Open vSwitch (OvS) is open-source multi-layer virtual switch, it can operate as software within another machine; such as inside a switch, router or virtual machine. It is used to design network automation through programmatic extensions while supporting standard management interfaces and protocols like: OpenFlow, NetFlow, sFlow, IPv6 and 802.1Q virtual LANS (VLAN).

The key components in OvS are the bridges, ports and flows. The bridges are like virtual switches that connect different virtual ports. In a virtual machine or container, these virtual ports created can be attached to the bridges allowing communication. These virtual ports are network interfaces that can connect to a VM or physical network interface. Open switch use flow entries to decide how network packets will be handled. These flow entries are rules that specify actions on how the switch should handle packets; either to forward, drop or modify them. Figure 6 below gives a summary of OvS.

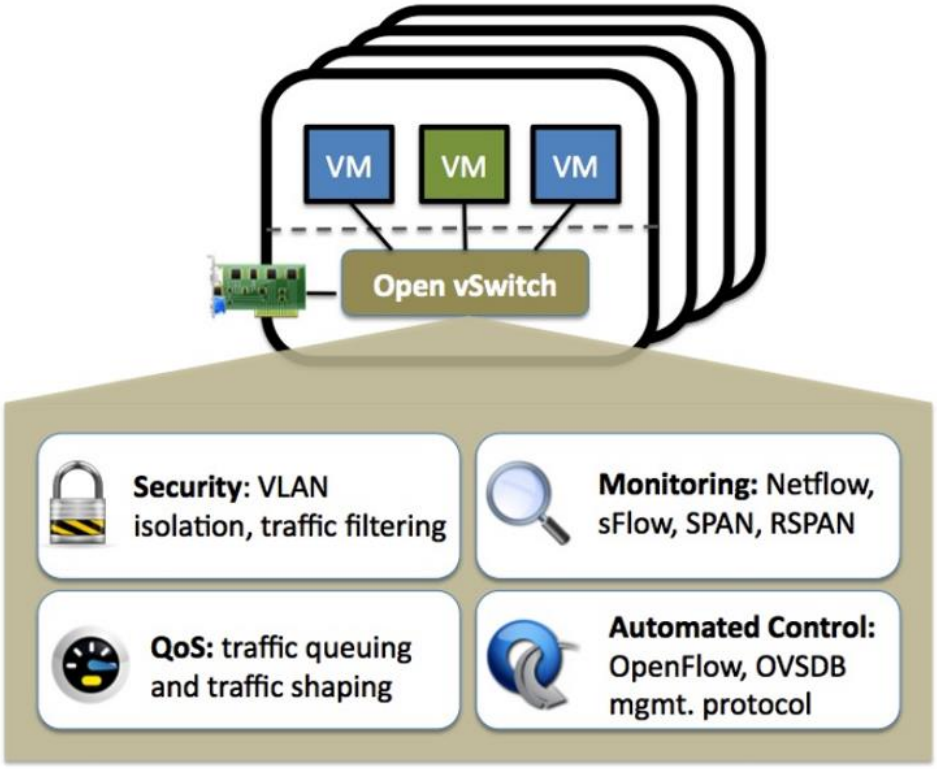

Figure 6: OvS features *[16]*

2.5 SDN Controllers

2.5.1 RYU Controller

RYU, pronounced as 'ree-yooh' is an open source SDN framework that allows users control OpenFlow enabled devices [19]. It is written in python and supports networking protocols such as NETCONF and OF-config [20].

Figure 7 shows a systematic view of RYU Controller's architecture. The application layer made up of user-facing programs like network and business logic sits at the top layer much like an SDN network, then we have the control layer which communicate with the application layer via APIs that developers can create depending on what sort of management they want. At the last layer is the physical layer which is made up of the switches and routers.

Ryu communicates the OF-config (OpenFlow configuration) down to this layer using the southbound APIs. Northbound Interfaces or APIs such as Quantum, Restful management or some user custom API are used to relay communication between the control layer and application layer as earlier stated [21].

Thanks to the innovation of SDN controllers, like Ryu, developers only need to focus on the logic of the flow in a network instead of trying to understand the syntax or details of OpenFlow protocol to configure network devices [22].

To install Ryu, you would use pip to run the command below and then run the Ryu manager:

```
>Sudo pip3 install ryu
>Ryu-manager –version
```

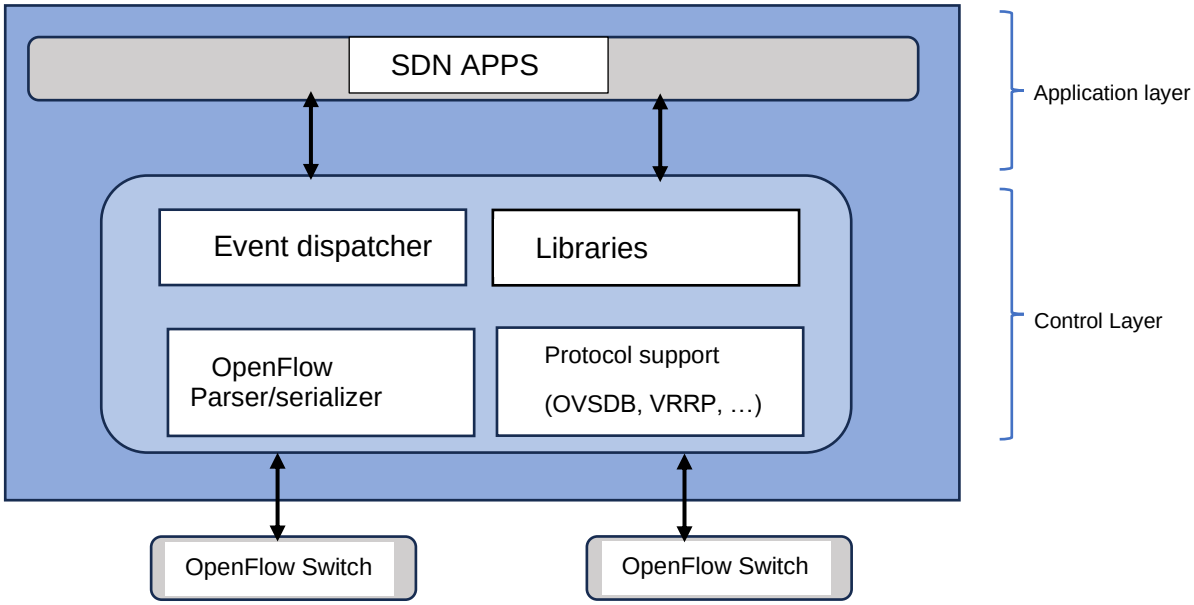

Figure 7: Ryu architecture

2.5.2 POX Controller

The POX controller is an open-source controller written in Python language and designed for use in SDN environments. POX started out as an OpenFlow controller, but can now also function as an OpenFlow switch, and can be useful for writing networking software in general.

To get started with POX, you clone its repository, set up your environment, and start the controller with the desired modules. To do this you first install by typing the command:

```
>git clone https://www. github .com/noxrepo/pox
```

Once done installing, you change directory to pox and then start the module you want. The *forwarding.l2_learning* module is what would be used in this project. It makes an OpenFlow virtual switch behave like

an Ethernet learning switch by storing source MAC address to switch ports and forwarding direct packets based on the learned MAC address table. You can get this started by using the command:

```
>cd pox
>~/pox/pox.py forwarding . l2_learning
```

2.5.3 Faucet Controller

This is an open-source controller also based on OpenFlow. Although it supports advanced features like IPV4/IPV6 routing, VLANS, and ACLs, it is also known for its simplicity, security and scalability. It is implemented in Python and can run on various platforms making it a versatile choice for network management. It can be installed using pip. However, before running, the yaml file has to be configured to reflect your network setup. Then finally can be started. See commands in respective order below:

```
>sudo pip3 install faucet
>sudo mkdir -p /etc/faucet
>sudo nano /etc/faucet/faucet.yaml
>sudo faucet -c /etc/faucet/faucet.yaml
```

2.6 Intrusion Detection and Prevention Systems

2.6.1 Intrusion Detection Systems (IDS)

An Intrusion detection system is generally described as a device or component that continuously monitors and identiies computer attacks [23]. Computer networks are built using security devices like firewalls, authentication systems, and virtual private networks (VPNs) to guard against threats. The issue however is that, these appliances frequently possess vulnerabilities and are often inadequate in defending against the ever-evolving threat landscape. This landscape includes attacks that are continually modified to exploit system weaknesses or flaws, hence the need for Intrusion Detection Systems to detect malicious activity, allowing for timely remediation.

An IDS can be implemented in different ways, the core requirement from it remains the same which is the detection of malicious attempts. According Liao et al., methodologies behind IDS technologies are classified into three major categories [24]:

Signature based detection: A signature is a pattern that corresponds to a threat or attack that is known. Signature based detection takes

patterns or strings and compares it against captured events to recognize possible intrusions [24].

Anomaly based Intrusion (AD): An anomaly represents an inconsistency or deviation from the norm. Normal or expected behaviour are captured to form profiles. These profiles could be developed for different attributes e.g. login attempts, or some performance metric. The AD compares normal profiles with observed events in attempt to recognize attacks.

Stateful Protocol Analysis (SPA): This is also called Specification-based Detection. The IDS here works to recognize and trace protocol states. A good example of this is pairing requests with replies, such that a reply without a request indicates an attack.

In deployment however, IDS are usually host or network based. Host based detects attacks against a particular host, whilst network based detects attacks against a specific network segment. The can either exists as hardware, software or a combination of both [25].

2.6.2 Intrusion Detection and Prevention System (IDPS)

An IDPS goes beyond just detecting malicious attempts but also adopts countermeasures to defend against them. It proactively inspects system traffic for malicious packets and blocks the offending IP or restricts its access to the valuable or vulnerable resources. An IDPS can be configured to perform any action, given the event. It could stop the attack by first identifying the offending user or device and disabling its session or network access. It could adjust the config of the router, switch or firewall as well, and by so doing, change the overall security environment of the network or host to disrupt the attack. Similar to IDS, the two main types of IDPS are the Host-Based Intrusion Prevention Systems and Network-Based Intrusion Prevention Systems

Often times, IDPS is mistaken for a firewall. This is quite understandable because they both function in a network security capacity. However, they are unique and quite different from each other largely because IDPS unlike firewalls can recognize external as well as internal threats. The common model also is that an IDPS is situated behind an organization’s firewall as a different layer of security [26].

2.7 Denial of Service Attacks

A Denial-of-service (DOS) attack is an attempt to overload a website or network, with the aim of degrading its performance or even making it inaccessible [27]. An adversary can achieve DOS attack in SDN

networks by creating several new flows with the aim of flooding the control plane, OpenFlow switches and SDN controller and as such, a legitimate user is unable to send packets across the network. Attackers are able to compound this attack by generating several new flows using spoofed IP addresses. According to Lubna et al., the OF switch buffers packet-in messages before sending them to the controller, the buffer fills up if several new flows are received in a very short time causing:

1) the entire packet of the new flow to be sent to the controller instead of flow's headers-only-packet

2) higher consumption of the Control plane's bandwidth resulting a delay installation of new flow rules.

Asides from the buffer getting filled up, the forwarding table of the OF switch could also be filled up, so when the controller sends a new flow rule to the switch, it is unable to install it and just sends an error message to the controller [28]. In any case, communication bandwidth, memory and CPU in both control and data plane are consumed making it difficult for requests from legitimate users to be processed.

Distributed Denial of Service (DDOS) attacks utilize many sources of attack traffic, while DoS utilizes a single connection. We will discuss the three attacks used in this project briefly below:

2.7.1 Port Scanning attack

Port scanning attack is a technique for discovering hosts' weaknesses by sending port probes [29]. By scanning for open or closed ports, attackers are able to identify what services are currently running on a host machine so as to potentially exploit vulnerabilities associated with those services. It can lead to a DoS attack.

2.7.2 SYN Flood Attack

A SYN flood attack is one that exploits the TCP three-way handshake process. In this attack, an attack node sends many Transmission Control Protocol SYN requests with spoofed source addresses to a node [30]. In a normal TCP handshake, a client sends a SYN request to a server, the server responds with a SYN-ACK, and the client completes the connection with an ACK. Here however, the attacker sends a large number of SYN packets (can be with spoofed IP addresses or not), but does not complete the handshake by sending the final ACK, leaving the server with several half-open connections.

2.7.3 ICMP Flood Attack

The Internet Control Message Protocol (ICMP) is used for sending various messages to convey network conditions. An ICMP flood happens when an attacker uses botnets to send large amounts of

ICMP packets to a target server in an attempt to exhaust bandwidth and prevent legitimate users from accessing the server [31].

2.8 Software & Tools

This section will briefly describe some additional software tools that would be used in this project.

2.8.1 Virtualization Software

A virtualization software allows a user to run two or more operating systems concurrently on a single PC. There are several available like Hyper-V, VMware, however, for the purpose of this project, we will be using **VirtualBox**. This is a widespread hypervisor software and gives all needed flexibility and efficiency to create a virtual host environment [32].

2.8.2 Python

Python programming language and all its supplementary libraries will be used in the development of this project. The SDN Controllers are built using python. The IDPS will be scripted using python programming language as well.

2.8.3 Wireshark

Wireshark is the world's foremost protocol analyser [33]. It is widely used to capture and analyse packets of data sent and received in the SDN virtual network or even a traditional network. The command for installing Wireshark is

```
apt-get install wireshark.
```

Other tools to be used include Network scanner tools like Nmap, and security testing tools like Hping3

3 Related Works

This chapter reviews key related works, emphasizing their contributions, limitations and how this project is different from their work. The works are organized around key themes- performance metrics for SDN controllers, detection techniques for IDPS, and lastly specific statistical detection methods- to provide a clear understanding of existing approaches and how this project builds on them.

3.1 SDN Controllers and Performance Metrics

Before delving into the work performed in IDPS, it is valuable to briefly examine SDN controller comparisons based on key performance metrics such as latency, scalability and throughput before factoring in security concerns.

Several studies have focused on this evaluation. For instance, Ola et al. conducted a comprehensive study comparing multiple controllers, including POX, RYU, and Floodlight, focusing on metrics like latency and scalability across different network topologies using Cbench as a testing tool [6]. Similarly, Neelam et al. extended this work by analysing these controllers in more complex scenarios, employing Mininet- a network simulator software [34]. In another variation, Fidha et al. conducted a performance analysis using Mininet WIFI platform mostly to evaluate which controller is the best in emulated wireless networks [7].

However, while these studies provide valuable insights into the network performance and efficiency, they largely overlook security-specific metrics, such as speed of detecting and mitigating cyberattacks, which are crucial for real-world applications and getting a comprehensive understanding of SDN controller's capabilities. This project fills this gap by comparing the security performance of controllers, particularly their ability to detect and mitigate threats.

3.2 Intrusion Detection & Prevention Systems (IDPS) in SDN

Several approaches have been explored in the literature, focusing on different attack vectors and detection techniques. These approaches can be categorized into reactive detection mechanisms, anomaly-based detection, and hybrid techniques. And as expected, they all have their strengths as well as limitations.

3.2.1 Reactive Detection Mechanisms

This method relies on identifying specific signatures of known attacks. SLICOTS, developed by Mohammadi et al. [35] demonstrates this approach by utilizing the OpenDayLight controller to detect SYN flood attacks. It was implemented as an extension module, monitoring ongoing TCP handshakes and installing short-term forwarding rules during the process. If a handshake completes successfully, a permanent rule is set; otherwise, the system blocks malicious requests when the number of half-open connections exceeds a threshold. While effective in reactive environments, its dependence on real-time flow installations introduces latency and limits its applicability in proactive security environments.

In a different take, Chin et al. proposed a technique for detecting SYN Flood attacks by deploying monitors in the network to inspect packet signatures [36]. The system uses correlators to query the flow tables of switches and block malicious traffic based on source IP addresses. However, this method's limitation is with scalability as requiring monitors for every switch makes it impractical for large-scale deployments, particularly in enterprise networks.

3.2.2 Anomaly-Based Detection Approaches

Anomaly-based systems focus on detecting deviations from normal network behaviour for detecting unknown attacks, or zero-day attacks. Wang et al. proposed a DoS detection mechanism using an anomaly-based module that matches attack patterns in a database with new packet flows [37]. While this method reduces dependence on static signatures, it introduces overhead by utilizing external applications for detection.

Abubakar et al. further criticized signature detection systems, stating their inefficiency in detecting zero-day threats [38]. Their anomaly-based approach uses traffic pattern analysis to identify and block suspicious flows in real-time. However, the risk of high false positives that tends to degraded network performance remains a common challenge with anomaly-based systems, especially if not properly tuned.

3.2.3 Hybrid Techniques: Threshold Algorithms

Hybrid detection techniques combine reactive and anomaly-based approaches, leading to enhanced detection accuracy while minimizing false positives. Threshold-based detection methods are commonly used to manage port scanning attacks, and flooding attacks like the SYN flood attack. Charles et al. developed a lightweight system that tracks packet-in messages and uses predefined thresholds to block traffic that exceeds normal limits [39]. Similarly, Daichi et al., applied

the spectral residual method to detect vertical port scans, whilst adjusting the flow rules dynamically based on packet analysis [40].

A more advanced hybrid system however, was proposed by Birkinshaw et al. who combined Credit-Based Threshold Random Walk (CB-TRW) and Rate Limiting (RL) to detect and mitigate DoS attacks [10]. By setting adaptive thresholds for TCP packets, the system was able to effectively detect abnormal traffic patterns without compromising legitimate traffic. Although, Dotcenko et al. [41] implemented a similar approach, they did not integrate real-time mitigation, a limitation addressed by Birkinshaw's work.

Looking ahead, there is growing interest in Entropy based detection [42] [43] [44] as well as leveraging machine learning and Artificial intelligence to further optimize IDPS in SDN environments [45] [46] [47].

Notwithstanding, a blend of machine learning with traditional detection techniques represent the future of IDPS in SDN, enhancing adaptability and precision.

3.3 Statistics based detection- IDPS

The effectiveness of an IDPS can sometimes hinge on the statistical methods employed for detecting and mitigating threats. In this report, we would be looking at the Credit-Based Threshold Random Walk (CB-TRW) and Rate Limiting (RL) methods.

Birkinshaw et al. in 2019 published an Intrusion Detection and Prevention system to defend against Port-Scanning and DoS attacks. The used the CB-TRW (Threshold Limiting) and RL (Rate limiting) method to identify the attacks. The algorithm was able to achieve this by taking the packet-in messages sent to the controller, and extracting the packets in it [48]. The packets (ICMP, UDP, SYN) are counted against a set related threshold. Any number of packets under this threshold is assumed to be legitimate traffic, whilst anything above it is tagged suspicious. This is based on the assumption attackers send large number of packets at a time to valuable TCP ports, therefore the IDPS tracks packets and would immediately report an anomaly if the connection rate from a host to a server exceeds the set threshold. Their experiment was carried out using POX controller, Open vSwitch and their results proved that the proposed mechanism can efficiently detect denial of service attacks.

Similarly, Dotcenko et al. implemented an IDS using CB-TRW and rate limiting algorithm validating their approach using Beacon controller and Mininet [41]. According to their work, diagnostic features of the network traffic is collected, and upon identification of suspicious traffic, the threat level is calculated and expressed on a three-level scale: low, medium or high. However, no mitigation is done; a limitation which

Birkinshaw's research overcomes. **Kandoi et al.** focused on using rate limiting by setting a limit to the number of packets sent to the controller such that if the controller detects that it is receiving more messages than it can handle, it installs a rule on the switches instructing them to reduce the rate at which they send messages [49]. The issue with this approach however is that it affects all flows, therefore legitimate traffic can be dropped.

Finally, **Mehdi et al.** implemented both the entropy algorithm and TRW-CB with rate limiting integrating it with an OpenFlow-compliant switch and NOX controller to detect anomalies in home-based networks [50]. Their system dynamically updated flow rules to block malicious traffic, demonstrating the potential of these combined statistical methods to improve detection accuracy and responsiveness in SDN environments

3.4 Summary

The reviewed literature highlights a gap in the evaluation of SDN controllers concerning their security capabilities, particularly in detecting and mitigating cyberattacks like SYN Floods, Port Scanning, and ICMP Floods. While many studies focus on performance metrics such as latency and throughput, the relationship between these metrics and the speed of attack detection using integrated IDPS scripts is underexplored.

This project aims to fill this gap by testing the speed of detection and mitigation of various SDN controllers (RYU, POX) when integrated with an IDPS script. By comparing these controllers based on their security performance, this research provides insights into their effectiveness in real-world applications.

Additionally, in smaller environments, using multiple controllers can be costly and lead to inconsistencies, making it essential to identify a controller that balances performance and security. This work highlights the importance of assessing SDN controllers not only for their communication efficiency but also for their responsiveness in threat detection and mitigation.

4 Methodology

This chapter discusses the processes involved in developing the IDPS used for this project, and the concepts they operate with. It also briefly highlights the controller selection and type of attacks performed in this project.

4.1 IDPS Design Overview

The IDPS proposed for this project uses a network-based IDS. It uses information from the packets that are received by the controller; the IDPS “listens” for Packet-In events and intercepts them for processing. Two connection-based techniques are used to form the anomaly detection system: TRW and RL. Both techniques have performed well in similar network security simulations [10].

The first thing the IDS checks is if the packet originates from a host that is in the blacklisted hosts. If it is, the packets are dropped immediately. Next, the number of packets received are analysed first to determine their packet type, either TCP, or ICMP packet. If it is none of these packet types, normal control plane processing continues. Where they are packet types that are of interest to us, it reads the configuration of the TCP/ICMP flags in the packet header, checks whether the communication between the host and the server are being tracked. If they are not being tracked, the IDPS starts to track them, noting the address of the host, the address of the server, the number of unsuccessful connection initiations attempted by the host (each connection is taken to be unsuccessful and later corrected if it turns out to be successful), the total number of TCP connections attempted by the host, and the time when the connection started to be tracked. The IDS then uses a customizable threshold to determine if a host exceeds the allowed limits. Where a host is guilty of this, it is added to the blacklisted MAC address list and the packets are dropped.

Whilst all this is happening, logs are written to take note of the time detection was made, mitigation effected and the time it took.

- Detection time is the time taken by the IDPS to detect an attack after it has started based on threshold set
- Mitigation time is the time taken by the IDPS to block the attacker after detection.
- Speed of detection is the difference between mitigation time and detection time.

4.1.1 Threshold Limiting and Rate Limiting (RL)

The IDPS design is based on two algorithms, the first being threshold limiting and the later rate limiting.

Threshold Limiting

The IDPS monitors the Packet-in traffic for all ongoing TCP handshake connections. With every successful handshake process completed between two users, a variable c is created with value 1 and incremented by 1 for any subsequent completed handshakes. If the number of connections exceeds a predefined threshold, then a forwarding rule for blocking traffic from the given host is installed.

Rate limiting

When a packet is received from a host, the time the first packet is received is stored. If that host sends packets that are more than the threshold set within a period of 60 seconds from the time the first packet is received then the IDS detects this as a policy violation and any subsequent traffic from that host is dropped. A timer is set to clear traffic that is older than 60 seconds and did not violate any network policies in order to prevent an attack where the adversary sends packets below the threshold in the allotted 60 seconds. This goes to ensure that normal benign traffic is not detected.

In this project, port scanning employs just Threshold Limiting while the flooding attacks use both threshold Limiting and Rate Limiting. The pseudocode for Syn flood and ICMP flood attack were written by me. The pseudocode for Port scanning was gotten from Birkinshaw et al. [10] however, changes were added to log detection and mitigation times. To ensure ease of understanding and reduce complexity, the Pseudocode for each attack is described below to show how the algorithm is employed in each attack:

SYN Flood IDPS.

```
1. SET THRESHOLD = 50
2. INITIALIZE BLOCKED_HOSTS as empty list
3. INITIALIZE SYN_TRACKER as empty dictionary

4. ON PacketIn:
   a. IF Packet.src in BLOCKED_HOSTS:
      i. DROP Packet
   b. IF Packet.type != TCP:
      i. ALLOW Packet
   c. IF Packet.is_SYN:
      i. IF Packet.src not in SYN_TRACKER:
         - ADD Packet.src to SYN_TRACKER with initial SYN count
      ii. ELSE:
         - IF SYN_TRACKER[Packet.src].time > 1 minute:
            - REMOVE Packet.src from SYN_TRACKER
            - RETURN
         - INCREMENT SYN_TRACKER[Packet.src].count
      iii. IF SYN_TRACKER[Packet.src].count > THRESHOLD:
         - LOG "Attack detected"
         - ADD Packet.src to BLOCKED_HOSTS
         - APPLY flow modification to block Packet.src

5. LOG detection time, mitigation time, and calculate speed of mitigation
```

Figure 8: Syn flood IDPS pseudocode

Port Scanning IDPS

```
1. SET PORT_SCAN_THRESHOLD = 100
2. INITIALIZE BLOCKED_HOSTS as empty list
3. INITIALIZE PORT_SCAN_TRACKER as empty dictionary

4. ON PacketIn:
   a. IF Packet.src in BLOCKED_HOSTS:
      i. DROP Packet
   b. IF Packet.type != TCP or Packet.flags not in [SYN, SYN_ACK, ACK]:
      i. ALLOW Packet
   c. IF Packet.is_TCP and Packet.is_relevant:
      i. IF Packet.src not in PORT_SCAN_TRACKER:
         - ADD Packet.src to PORT_SCAN_TRACKER with scanned port info
      ii. ELSE:
         - IF PORT_SCAN_TRACKER[Packet.src].time > 1 minute:
            - REMOVE Packet.src from PORT_SCAN_TRACKER
            - RETURN
         - INCREMENT PORT_SCAN_TRACKER[Packet.src].port_count
      iii. IF PORT_SCAN_TRACKER[Packet.src].port_count > PORT_SCAN_THRESHOLD:
         - LOG "Attack detected"
         - ADD Packet.src to BLOCKED_HOSTS
         - APPLY flow modification to block Packet.src

5. LOG detection time, mitigation time, and calculate speed of mitigation
```

Figure 9: Port scan IDPS pseudocode

ICMP IDPS

```
1. SET ICMP_THRESHOLD = 50
2. INITIALIZE BLOCKED_HOSTS as empty list
3. INITIALIZE ICMP_TRACKER as empty dictionary

4. ON PacketIn:
   a. IF Packet.src in BLOCKED_HOSTS:
      i. DROP Packet
   b. IF Packet.type != ICMP:
      i. ALLOW Packet
   c. IF Packet.is_ICMP:
      i. IF Packet.src not in ICMP_TRACKER:
         - ADD Packet.src to ICMP_TRACKER with initial ICMP count
      ii. ELSE:
         - IF ICMP_TRACKER[Packet.src].time > 1 minute:
            - REMOVE Packet.src from ICMP_TRACKER
            - RETURN
         - INCREMENT ICMP_TRACKER[Packet.src].count
      iii. IF ICMP_TRACKER[Packet.src].count > ICMP_THRESHOLD:
         - LOG "Attack detected"
         - ADD Packet.src to BLOCKED_HOSTS
         - APPLY flow modification to block Packet.src

5. LOG detection time, mitigation time, and calculate speed of mitigation
```

Figure 10: ICMP IDPS pseudocode

While there may be slight differences in the actual code across each controller due to library, modularity and mode of operation; their core operationality follows these pseudocodes. The generic flowchart is given below:

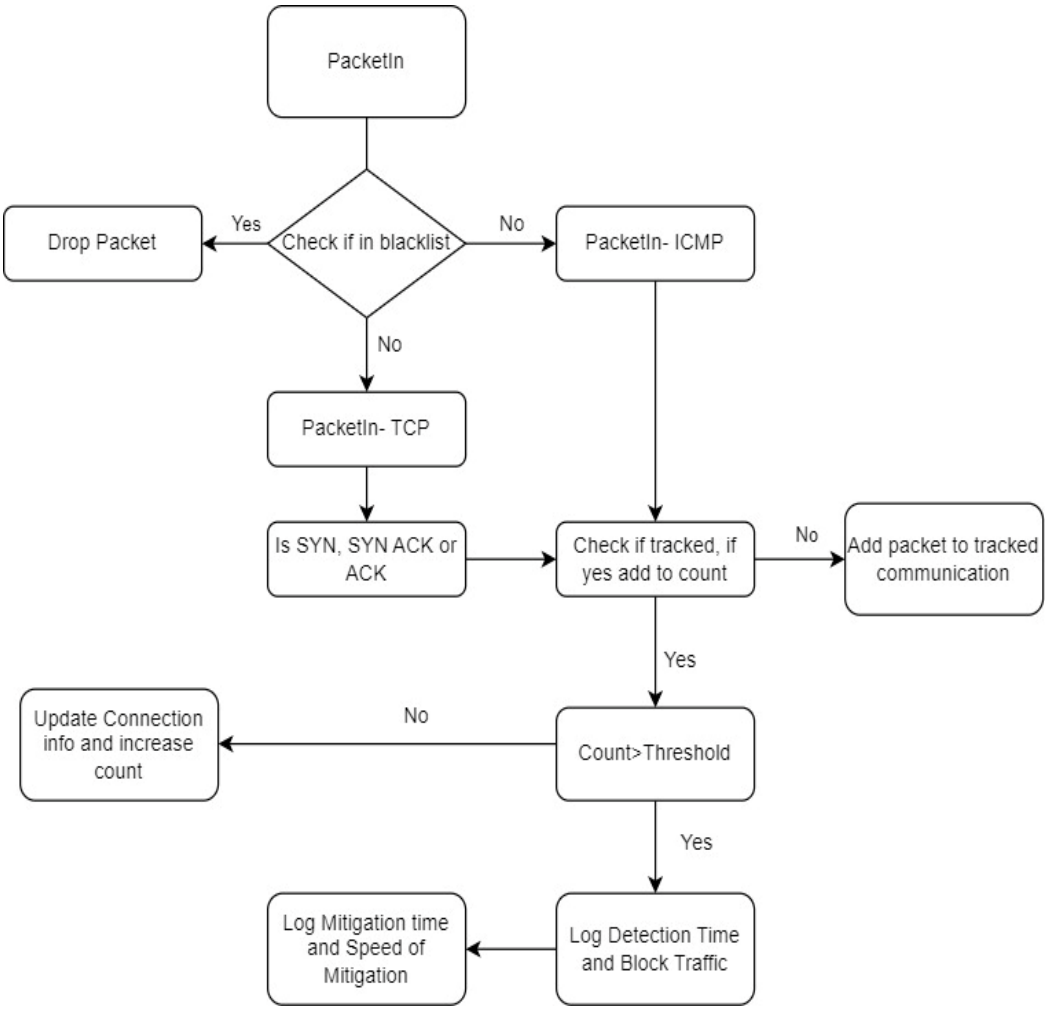

Figure 11: IDPS flowchart

4.2 Attacks

Three attacks were used to test the IDPS and take required measurement. These are Port Scanning, TCP Syn flood, and lastly ICMP flood attack. Port scanning is a common reconnaissance technique essential for identifying open ports and vulnerabilities they might have. Detecting port scans helps prevent serious attacks. SYN flood and ICMP flood attacks are widely used DoS attacks that disrupts services. Testing against these attacks shows a controller's ability to handle high-volume traffic and maintain service availability.

Port Scanning and SYN flood employ the TCP protocol, however, to add a layer of depth and rigour in terms of attack vector, ICMP attack

was also used. It targets a different protocol- ICMP, giving a broader assessment of the IDPS's capabilities and SDN controller's effectiveness in assisting to mitigate.

For detailed analysis, impact on legitimate traffic whilst these attacks are running will also be tested.

4.3 SDN Controllers

POX is lightweight controller often used in research, providing a baseline for comparing more advanced controllers. Ryu on the other hand is modular an event-driven and scalable. It is ideal simulating a more real-world environment with complex networks and advanced features. Faucet is similar to Ryu but employed a lot more in enterprise networks and adopted for its production-ready capabilities. It also supports modern network features like IPv6 and VLANs.

Choosing these controllers allowed to properly compare controllers across research, corporate and enterprise realm to give a comprehensive analysis.

5 Testbed Design and Implementation

The testbed machine is a server with 16GB RAM, an Intel Core i7-7700 CPU and a 64-bit OS, running Ubuntu 20.04.6 LTS Desktop. Experimentation is performed using the POX, Ryu, controllers, OpenFlow 1.0, OvS 3.4.90, and four guest hosts (Virtual machines). All the virtual machines run Ubuntu 20.04.6 LTS and Ubuntu was chosen because it is one of the most popular Open Source based Linux distributions with strong community support as well [51].

5.1 Open vSwitch (OvS) Setup

Before installing, ensure you check if the kernel version of the linux machine that you're running supports OvS. Typically, when using a recent version of Ubuntu and OvS build like used in this project, the kernel should be supported. However, we can confirm by running:

```
Uname -r
```

And then search if your kernel version is supported using this link: http://docs.openvswitch.org/en/latest/faq/releases/ else there will might need to install an older kernel that is supported using this tutorial [52]. Ensure you boot into the kernel you've just installed before going further in the installation.

5.1.1 OvS Installation

The installation of OvS is taken from the official documentation from Open vSwitch [53]. You should ensure that you are signed in as root. This installation can be found in Appendix A.

5.1.2 OvS Configuration

After installation, also listed in Appendix A are the commands and instructions to configuring our virtual switch for the test environment [53]. For this project, an OvS bridge is created and named 'ovsbridge'. This is to allow us to direct all traffic going in and out of our virtual switch through one point. It will enable us monitor flows properly.

Once configuration is completed, ensure the OvS bridge can ping external networks by pinging google DNS and getting back connection response:

```
>Ping 8.8.8.8
```

Also, you can verify port mapping on the OvS bridge and check mac information respectively using command below:

```
>OvS-dpctl show
>OvS-appctl /fdb/show OvSbridge
```

However, this might not be necessary now until after the VMs have been spun and connected to the virtual ports.

These OvS commands as well were utilised within the project:

- Print all flow entries: `OvS-ofctl dump-flows OvS-br`
- Delete all flow-entries: `OvS-ofctl del-flows OvS-br`
- Delete a specified flow-entry: `OvS-ofctl del-flows OvS-br <flow name>`
- OpenFlow features and port descriptions: `OvS-ofctl show OvS br`

It is also important to note that once the OvSwitch is turned off, the configuration would be cleared. You should repeat just the OvS Configuration (not the installation) listed in Appendix A, or you can put it in a bash script and run it upon startup and it should be properly setup again.

Extra Tools used: Hping3, Nmap

These tools do not have any apparent limitations that negatively affect the experiments. They are simple, straightforward and perform the basic required functions.

6 Experiments and Results

In this chapter, the IDPS is tested and its performance evaluated in multiple experiments. We also discuss results and make a critical comparison.

6.1 Environment and VM Setup

The network configuration for the experiments, has the OvS running on the host server, with Controller-VM, Attacker-VM, Victim-VM and Traffic-VM running on VirtualBox. The underlying networking is bridged to ensure communication amongst them. The VMs are connected to the OvS via virtual ports as illustrated in the Testbed Design Chapter. See diagram below for the full lab setup:

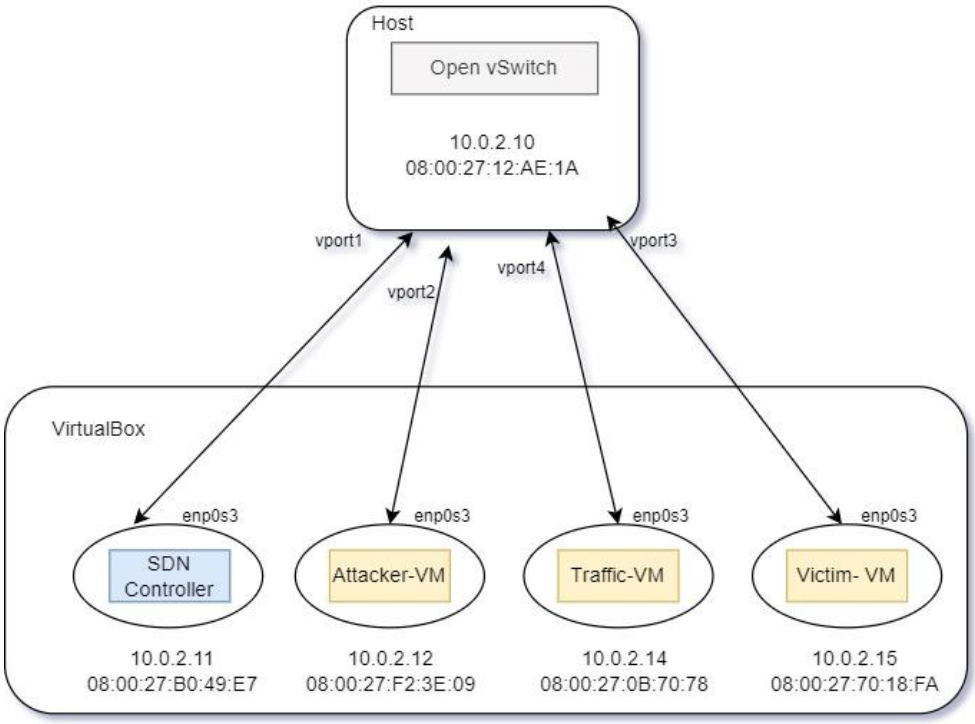

Figure 12: Virtual Network Configuration

A bridge (OvSbridge) was created in OvS alongside virtual ports. The virtual ports are brought up then we boot up the virtual machines and statically assign IPs on those virtual ports.

We ensure network connectivity by pinging each VMs before finally setting the SDN controller of choice as the controller in OvSbridge and setting fail mode to ‘secure’. This is to ensure that no traffic passes through the switch without the controller being up and processing it.

To measure impact on legitimate traffic, the Victim-VM runs a simple HTTP server. This is done during all experiments:

```
Sudo python3 -m http.server 80
```

The Traffic-VM then runs a script that sends HTTP GET requests to the victim Web-server every second and measures RTT time as well as time the requests were sent. This script can be found in Appendix B.

6.2 Experiment 1: Nmap Port scanning attack

6.2.1 POX Controller

This first experiment runs a python based IDPS that implements TRW and RL to detect a port scanning attack and mitigate the attack. It does this by installing a flow-entry in the virtual switch to drop all traffic from the Ethernet address of the identified attacker. The MAC blocker function ensures that further packets from the attacker is stopped. The IDPS Script is a modified version of Birkinshaw et al. script [10] , it was modified to show detection time, mitigation time, and finally calculate speed of mitigation.

The threshold for the number of TCP [SYN] packets that a host can send without the appropriate response was set arbitrarily (to hundred), further experimentation is required to tune the IDPS. The script for this is contained in Appendix C, and it contains doc strings and comments for readability.

The code for all experiments are in the Appendix section, they can also be found on github- https://github.com/mannylogic/SDN_IDPS/.

Also note that to ensure that pox has access to this, the script is saved in the directory /pox/ext/. The routing component used in this experiment is the l2_learning, a virtual learning switch which we enlist alongside our IDPS script using the command:

```
>~/pox/pox.py forwarding.l2_learning idpspox >
idpspox_log.txt 2>&1
```

The log is exported to idpspox_log.txt and ‘2>&1’ is to ensure that errors are logged as well.

Finally, the Attacker-VM does a port scan on the Victim-VM using nmap:

```
>Sudo nmap 10.0.2.15
```

6.2.2 Result

The results of the experiment shows that although the attacker attempted an Nmap port-scan on the victim, the IDPS worked as programmed, and most importantly the analysis shows the time taken for detection and prevention.

The Wireshark IO-graph below shows a spike in network activity indicative of the port scan attack.

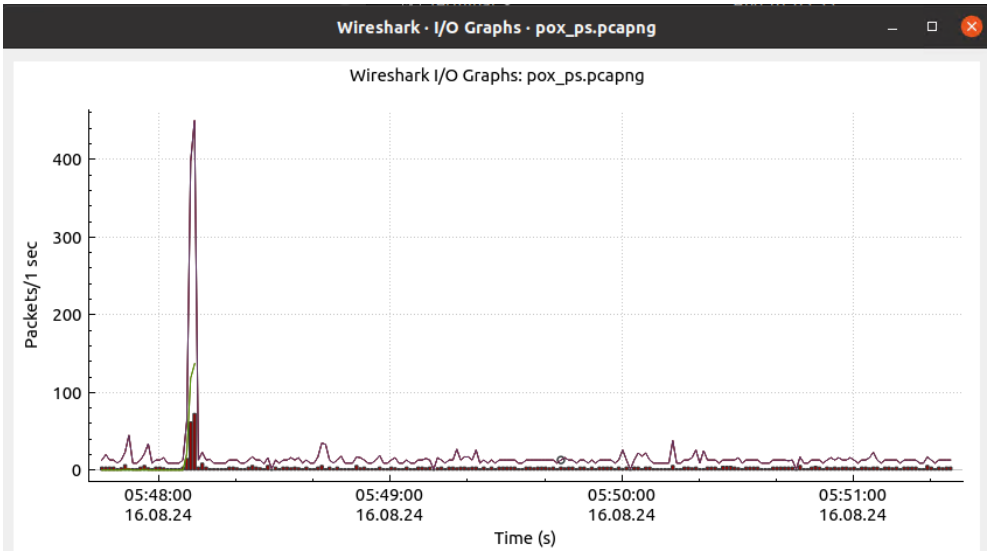

Figure 13: Experiment 1- POX wireshark graph

Going over to Wireshark and examining the captured packet file, we see the arrival time of the first packet from the attacker to victim (at the OvSbridge interface) marking the start of the attack as 05:48:08.169952637. However, the log from the IDPS shows that the attack was detected at 05:48:09.509640. See Figure 16.

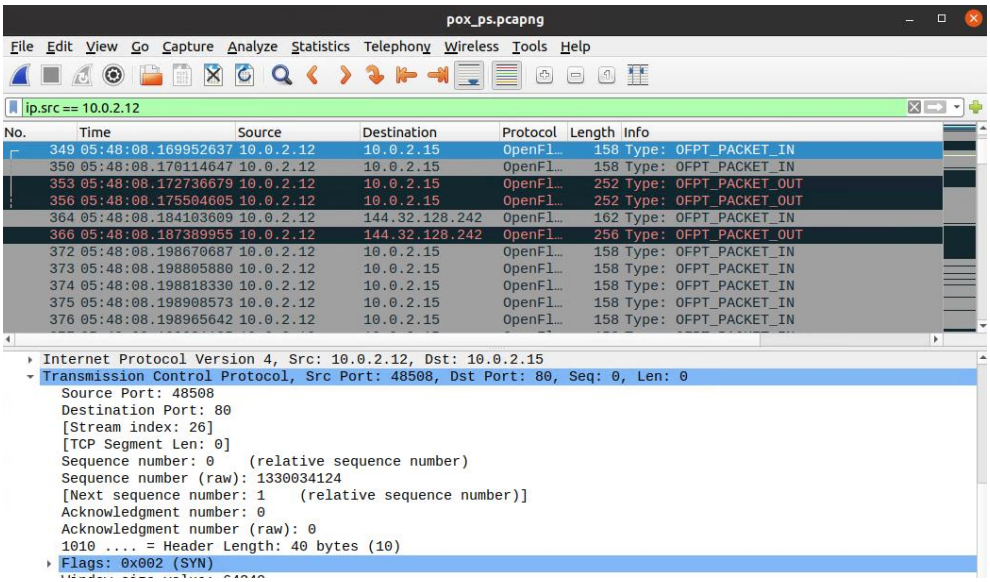

Figure 14: Exp 1: Wireshark packet analysis for POX

The attacker's terminal in Figure 15 shows that a subsequent attempt to scan the victim server quickly failed, returning no information about the server to the attacker, except that the host may not really be down, but blocking the ping probes. This shows that subsequent traffic from the attacker has been blocked.

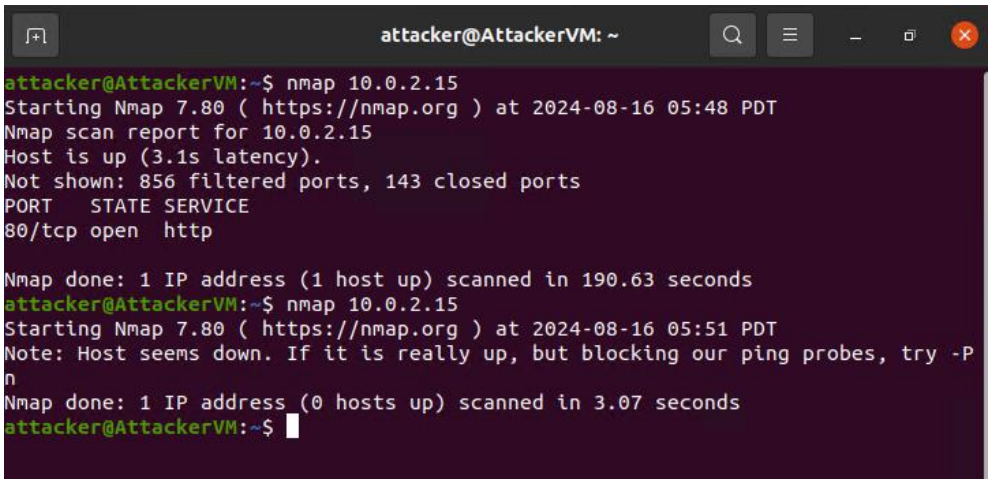

Figure 15:Exp 1- POX: Attacker terminal

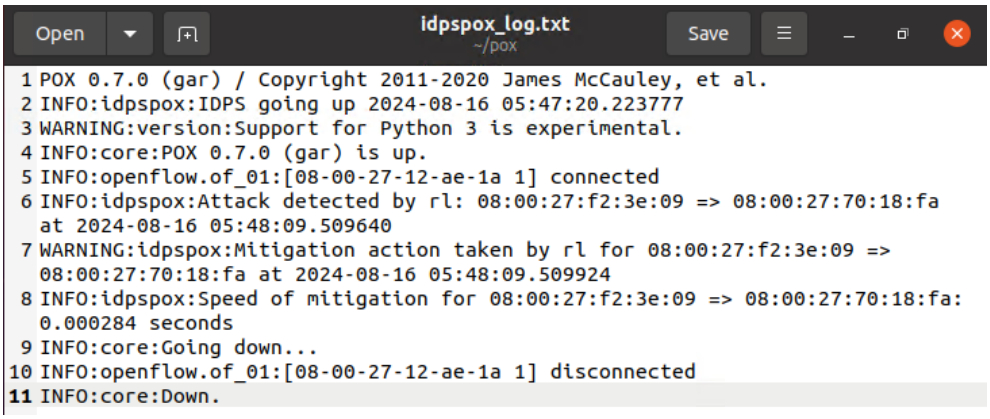

Figure 16: Exp 1: IDPS Log file from POX

Lastly, results from the IDPS log above are represented in table below.

Detection time	Mitigation time	Speed of Mitigation
05:48:09.509640	05:48:09.509924	0.000284 seconds

Table 1:Exp 1-Results: POX

6.2.3 Ryu Controller

Nmap port scanning was also carried out on Ryu controller to test speed of mitigation. The IDPS script (idpsryu.py) was written specially for this controller as its library is different as well as the way it handles events. Ryu follows a clear event-driven architecture enabling it to be more modular than POX. The script was saved in a folder in the home directory (ap_ryu) and ran like an application. See script in Appendix B.

The script was written to have switch forwarding capabilities like a simple switch so as to forward benign traffic, but also have the IDPS functionality to it. This is because unlike POX, Ryu has to be run with one app, hence the app has to perform everything required.

The attack was initiated using command:

```
 >sudo ryu-manager ~/ap_ryu/idpsryu.py >
idpsryu_log.txt 2>&1
```

The VM setup is the same as in the POX controller. VMs were restarted to clear any backlog from previous experiment.

6.2.4 Result

The results of the experiment shows that although the attacker attempted an Nmap port-scan on the victim Web server, the IDPS

worked as programmed, and most importantly the analysis shows the time taken for detection and prevention.

The Wireshark IO-graph below shows a spike in network activity indicative of the port scan attack.

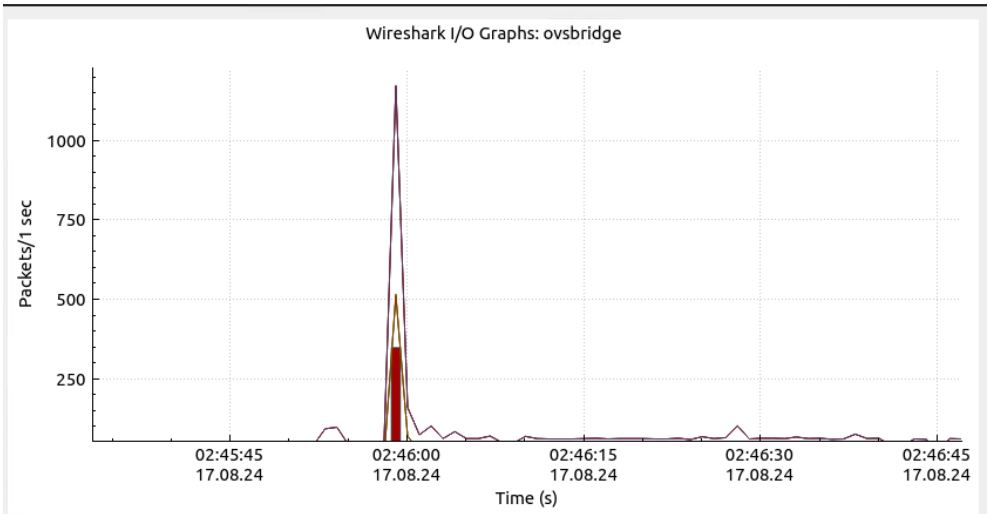

Figure 17: Exp 1- Wireshark I/O graph-Ryu

The attacker's terminal in Figure 18 also show that the attack was mitigated, and attacker host blocked from sending further traffic.

```
attacker@AttackerVM:~$ sudo nmap 10.0.2.15
[sudo] password for attacker:
Starting Nmap 7.80 ( https://nmap.org ) at 2024-08-17 02:45 PDT
Nmap scan report for 10.0.2.15
Host is up (0.034s latency).
Not shown: 537 closed ports, 462 filtered ports
PORT   STATE SERVICE
80/tcp open  http
MAC Address: 08:00:27:70:18:FA (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 241.21 seconds
attacker@AttackerVM:~$ sudo nmap 10.0.2.15
[sudo] password for attacker:
Starting Nmap 7.80 ( https://nmap.org ) at 2024-08-17 03:02 PDT

attacker@AttackerVM:~$ sudo nmap 10.0.2.15
[sudo] password for attacker:
Starting Nmap 7.80 ( https://nmap.org ) at 2024-08-17 03:51 PDT
Note: Host seems down. If it is really up, but blocking our ping probes, try -P
n
Nmap done: 1 IP address (0 hosts up) scanned in 0.50 seconds
attacker@AttackerVM:~$
```

Figure 18: Exp 1- Attacker terminal: Ryu

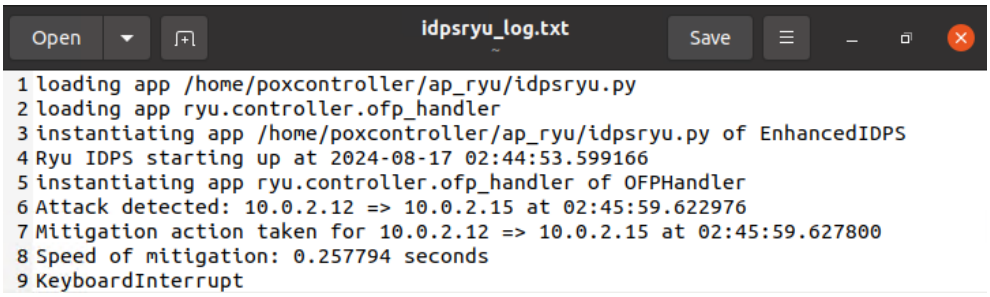

Figure 19: Exp1- IDPS log: Ryu

Lastly, results from the IDPS log above (fig xx) are represented in table below.

Time of Detection	Time of Mitigation	Speed of Mitigation
02:45:59.622976	02:45:59.627800	0.257794

Table 2: Exp 1-Results: Ryu

It appears here that Ryu speed of mitigation is slower than that of POX from this experiment.

6.3 Experiment 2: TCP SYN Flood attack

6.3.1 POX Controller

This experiment is to perform a SYN Flood attack on the Victim-VM, ensure the attack is mitigated and observe time taken to mitigate it. The IDPS script used here is bespoke, written to track TCP SYN request rates per connection and detects floods based on the threshold (currently set at 50). When a flood is detected, it blocks the source MAC address by installing a high-priority flow rule on the switch, and also log detection time, mitigation time, and finally calculate speed of mitigation. The script for this (tcppox.py) is contained in Appendix D, and it contains doc strings and comments for readability. Recall, the script is to be saved in the directory-/pox/ext/

The VM setup remains the same. However, the VMs were restarted to ensure no backlog from the previous experiments.

The attack was initiated using the following command:

```
>sudo hping3 -c 70 -S 10.0.2.15 -p 80
```

This uses the hping3 tool to send 70 SYN packets on port 80 to the Victim-VM

6.3.2 Result

The Experiment sent 70 SYN packets to the victim's IP on destination port 80. Recall that the threshold was set at 50 packets, the Figure below shows that 20 of the SYN packets failed. An attempt to attack the victim again didn't succeed as the MAC address of the attacker had been blocked. Hence the IDPS functioned as programmed, and most importantly the time taken for detection, mitigation and speed was logged as well. See Figure 21.

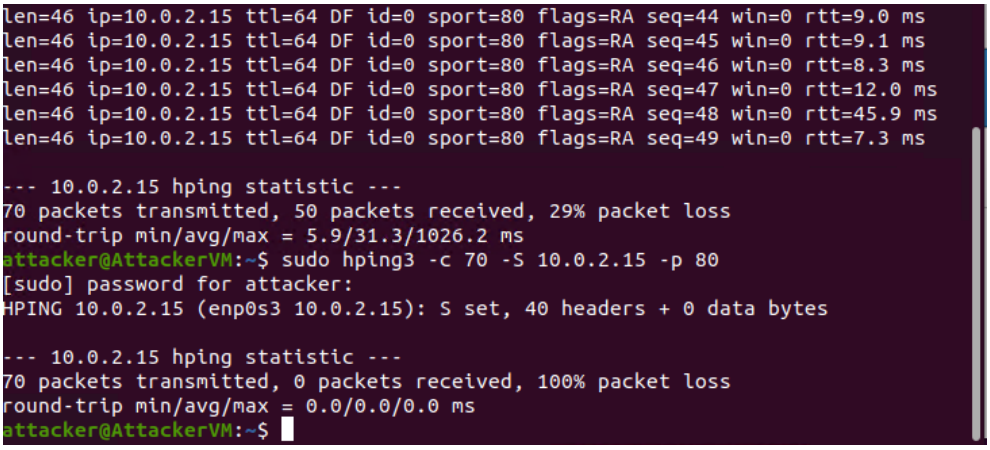

Figure 20: Exp2-Attacker terminal: POX

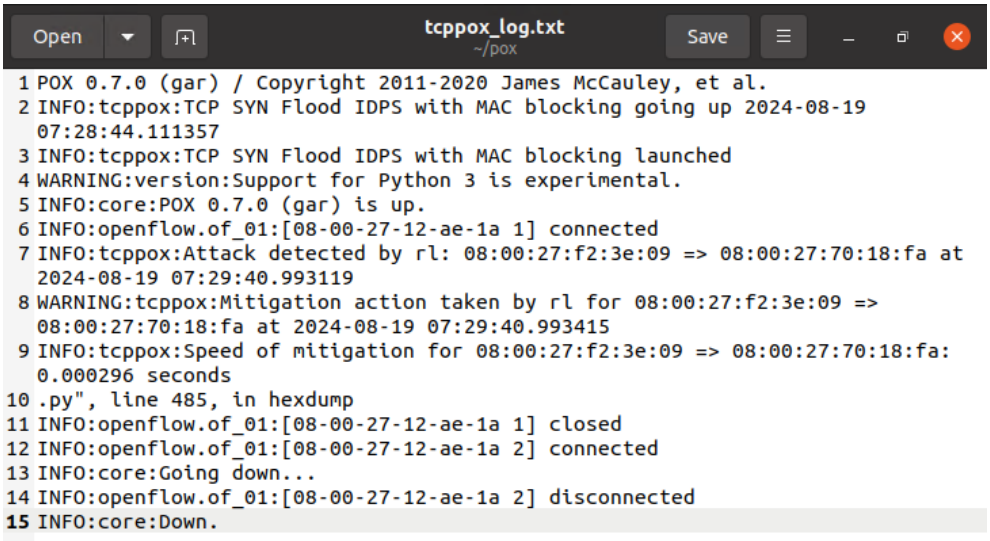

Figure 21: Exp2- IDPS log: POX

The IDPS table below shows the time taken for detection, prevention and speed of mitigation.

Time of Detection	Time of Mitigation	Speed of Mitigation
07:29:40.993119	07:29:40.993415	0.000296

Table 3: Exp2- Results:POX

6.3.3 RYU Controller

We restart the VMs again, and use Ryu as the controller. At this stage to ensure clear distinction between the SDN controllers, I configured the controller setting in the OvSbridge to listen on port 16633 instead of 6633. This is to ensure that moving forward, POX and Ryu run on different ports.

The IDPS script tcpryu.py was written to mitigate the SYN flood attack with the same threshold of 50 and log the times of interest. The script was adjusted for Ryu libraries and functions, however the IDPS algorithm remained the same. Also, the simple switch logic was embedded in the script to enable processing of benign traffic.

The IDPS was initiated using:

```
>sudo ryu-manager --ofp-tcp-listen-port 16633
~/ap_ryu/tcpryu.py > tcpryu_log.txt 2>&1
```

The attack was initiated using command:

```
 >sudo ryu-manager --ofp-tcp-listen-port 16633
~/ap_ryu/tcpryu.py > tcpryu_log.txt 2>&1
```

6.3.4 Results

As with the POX experiment, only 50 packets were transmitted. The IDPS functioned as programmed, and most importantly the time taken for detection, mitigation and speed was logged as well as can be seen in the log below:

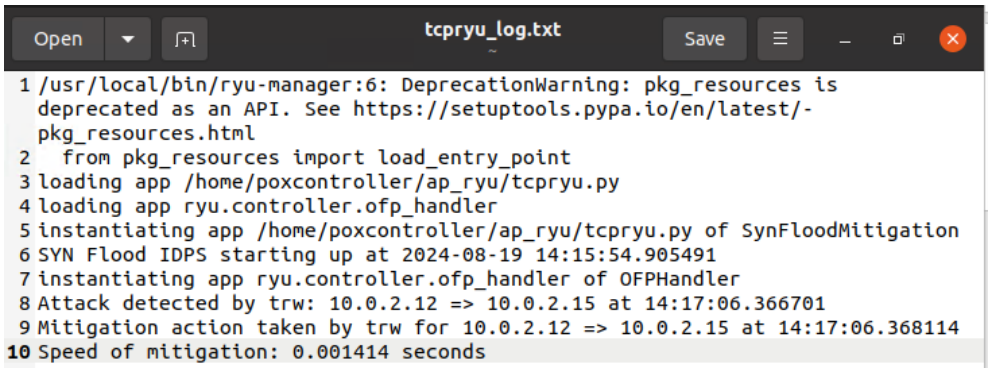

Figure 22: Exp2- IDPS log:Ryu

The IDPS table below shows the time taken for detection, prevention and speed taken.

Time of Detection	Time of Mitigation	Speed of Mitigation
14:17:06.366701	14:17:06.368114	0.001414

Table 4: Exp 2-Results: Ryu

6.4 Experiment 3: ICMP Flood attack

6.4.1 POX Controller

To add some depth for our analysis (targeting network layer), ICMP flood attack is also performed by sending a flood of ICMP packets to the victim's IP and tweaking the IDPS to detect, mitigate and log speed of mitigation. The IDPS can be found in Appendix E. The threshold used here is 50.

On the POX controller we enter command:

```
 >~/pox/pox.py forwarding.l2_learning icmppox >
icmppox_log1.txt 2>&1
```

And on the Attacker-VM we enter command:

```
 >Sudo hping3 -1 -d 120 --flood 10.0.2.15
```

'-1' specifies ICMP mode, '-d' 120 sets the data size of each packet to 120 bytes, and '--flood' sends packets as fast as possible, this mimics the realistic rate of a ping attack.

6.4.2 Results

The IDPS functioned as expected and the log below shows the times of detection, mitigation and speed of detection.

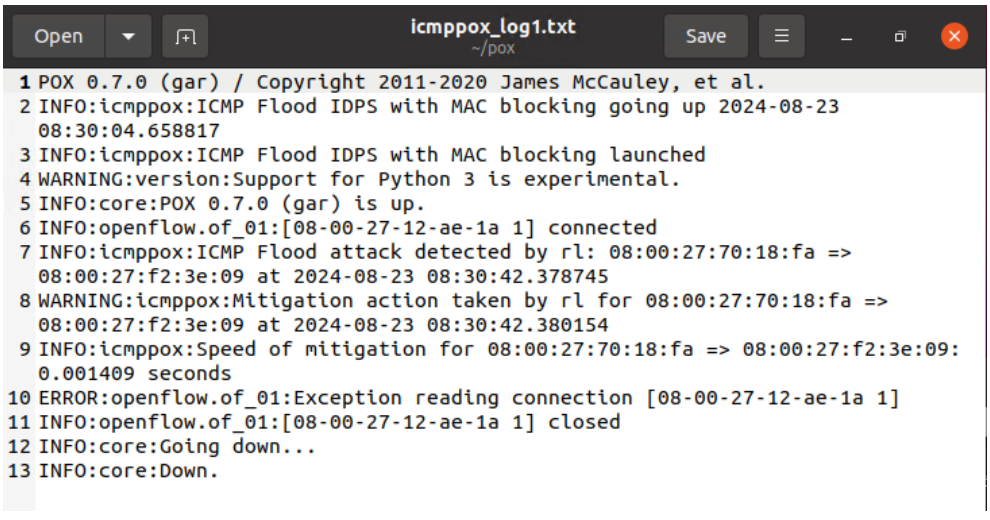

Figure 23: Exp 3- IDPS log:POX

The IDPS table below shows the time taken for detection, prevention and speed taken.

Time of Detection	Time of Mitigation	Speed of Mitigation
08:30:42.378745	08:30:42.380154	0.001409

Table 5: Exp 3- Results: POX

6.4.3 RYU Controller

The IDPS was again adjusted to leverage the libraries of Ryu mostly for packet parsing, blocking the mac address of suspicious hosts and performing the logic of a basic layer2 switch. The threshold was left at 50, and the IDPS can also be found in Appendix E.

On the controller, we enter the command below and use the same attack command as used in POX.

```
 >sudo ryu-manager --ofp-tcp-listen-port 16633
~/ap_ryu/icmpryu.py > icmpryu_log.txt 2>&1
```

6.4.4 Result

The IDPS functioned as expected and the log below shows the times of detection, mitigation and speed of detection.

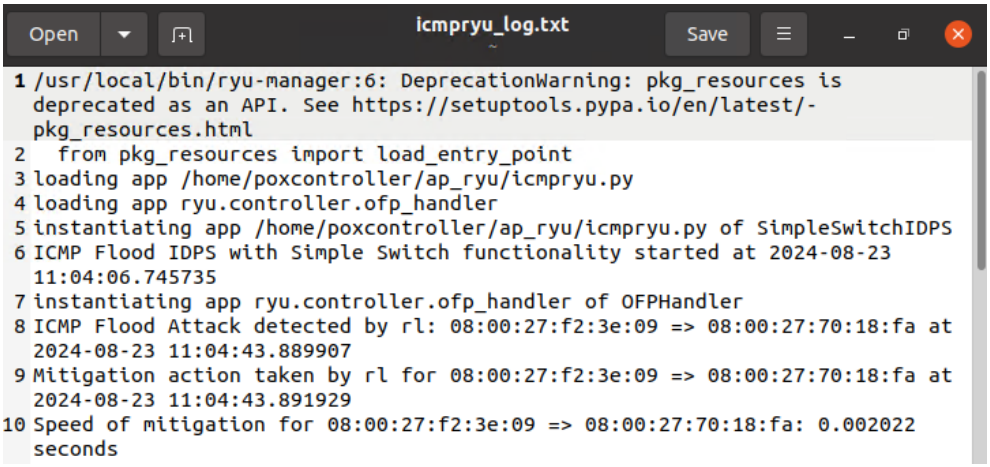

Figure 24: Exp 3- IDPS log: Ryu

The IDPS table below shows the time taken for detection, prevention and speed taken.

Time of Detection	Time of Mitigation	Speed of Mitigation
11:04:43.889907	11:04:43.891929	0.002022

Table 6: Exp 3- Results: Ryu

6.5 Impact on legitimate traffic using Round-trip Time (RTT)

To measure impact on legitimate traffic, we record Round-trip Time whilst the IDPS wasn't running and then whilst running during the experiments. Test Round-Trip-Time is the RTT while the IDPS isn't running, and no attacks are running as well. The other graphs show what the RTT was at the time each of those experiments were carried out to see what the impact of the IDPS was.

The results are presented below:

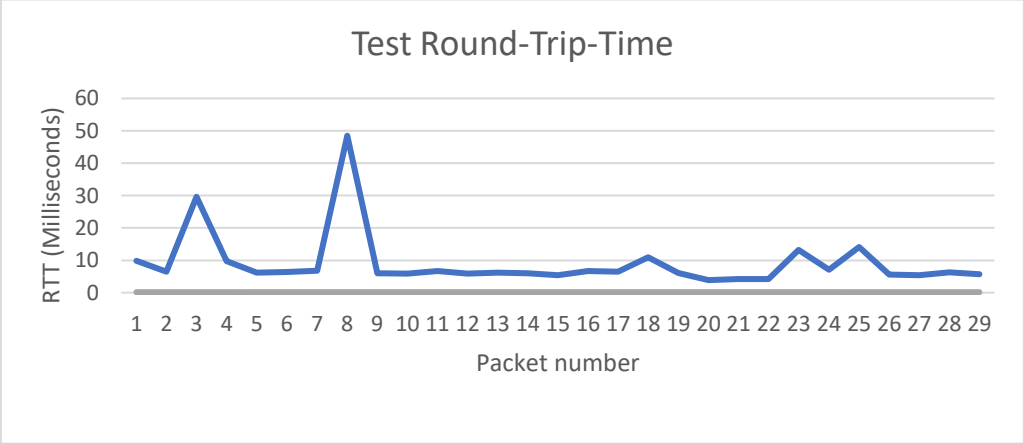

Figure 25: Test RTT

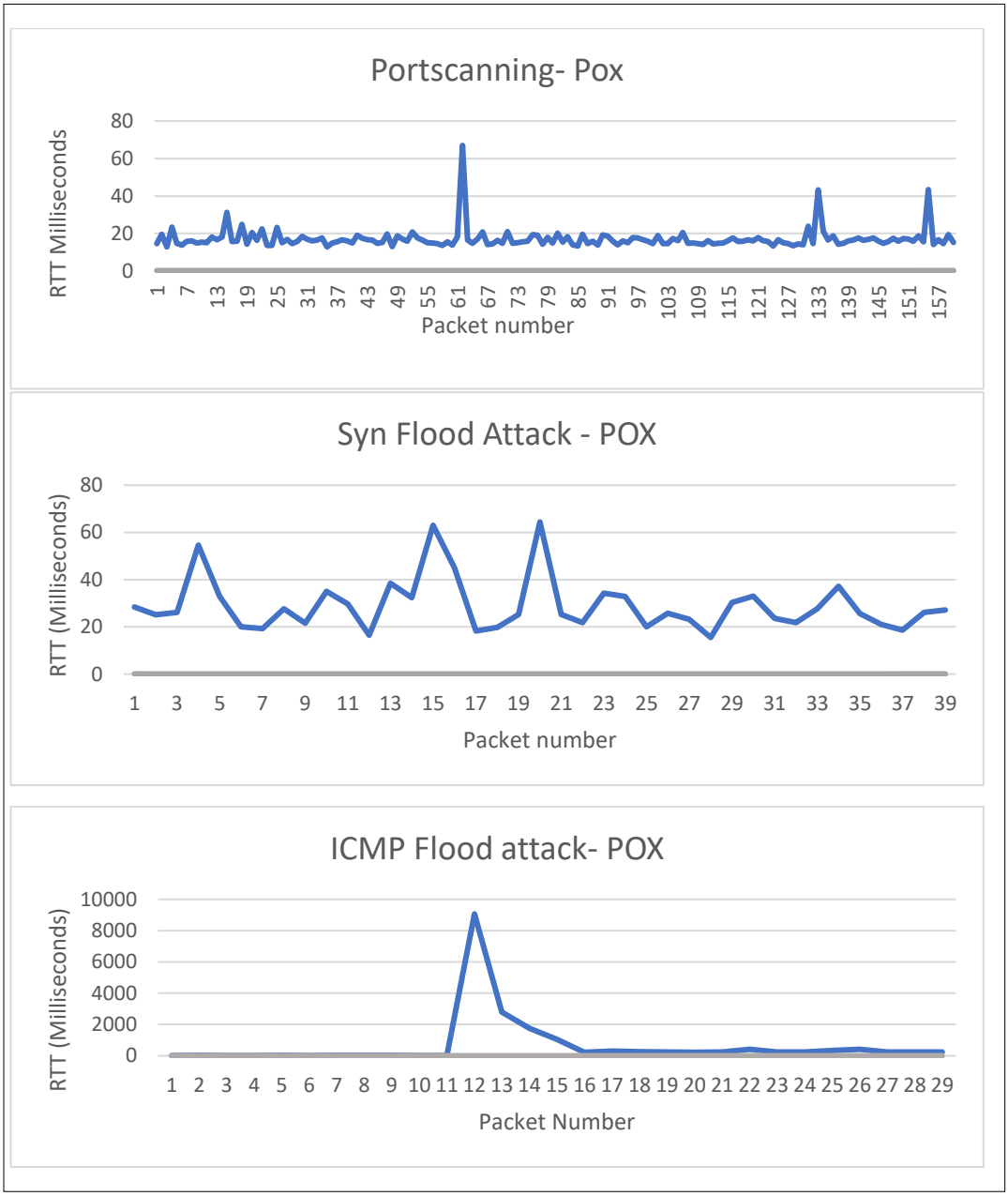

Figure 26: POX RTT

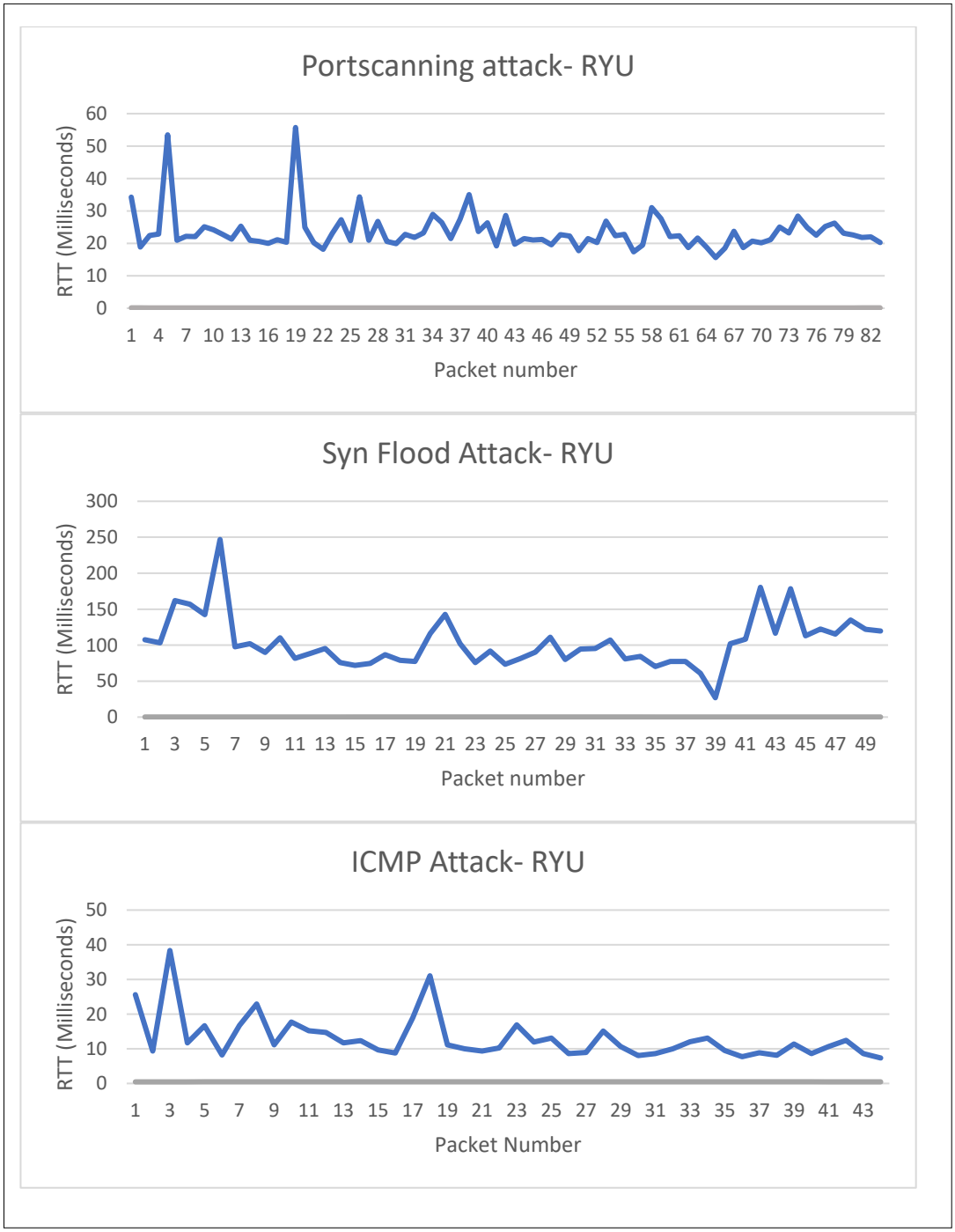

Figure 27: Ryu RTT

General Analysis on the experiments is given in the next section.

6.6 Comparison

The experiments conducted on the POX and Ryu controllers under various attack scenarios provide a comparative insight into their performance from a security perspective.

6.6.1 Speed of Detection and Mitigation

In the Port scanning attack, the POX controller outperformed the Ryu controller in detecting and mitigating the Nmap port scanning attack. Ryu had a slower mitigation speed of 0.257794 seconds or 257ms against POX's 0.28ms. In the SYN flood attack, there is a much narrower gap between both controllers. Even though both controllers effectively handled the attack, POX remained faster with mitigation speed of 0.29ms against Ryu's 1.41ms.

In the ICMP flood experiment, the performance of both controllers was nearly identical. POX controller had a mitigation speed of 1.41ms while Ryu had slightly better time of 1.40ms, indicating that both controllers handle ICMP flood attacks with similar efficiency.

6.6.2 Impact on Legitimate Traffic

Both controllers caused slight delays when the IDPS was active, but the magnitude of the delay varied depending on the attack.

For POX controller, RTT increased during flood attacks- most significantly during ICMP, but remained relatively stable during the port scanning experiment. It is easy to make a calculated assumption that POX's faster response time to both flood attacks contributed to less noticeable degradation in legitimate traffic performance.

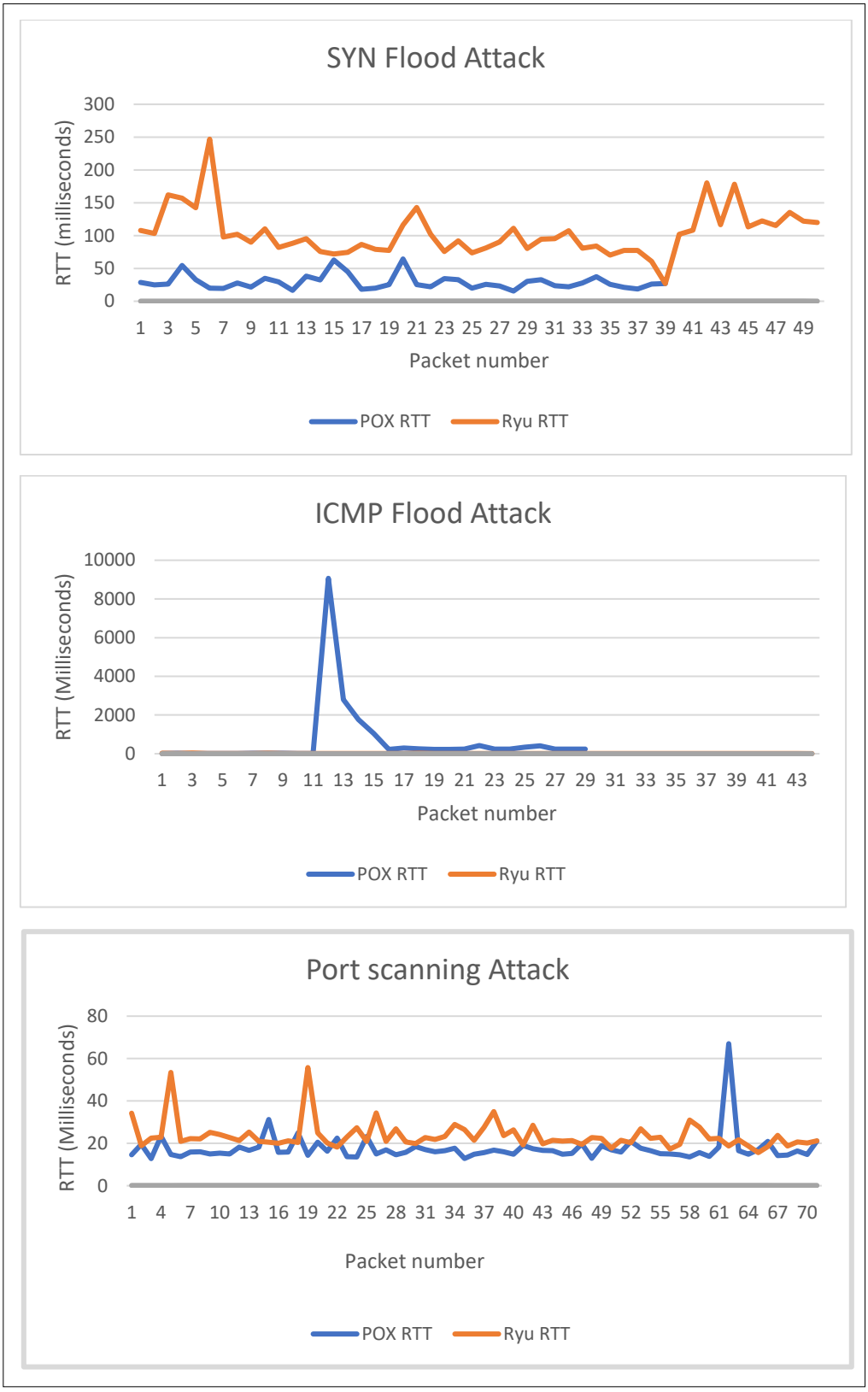

Figure 28: RTT comparison

With Ryu Controller on the other hand, we see that RTT increased more significantly during the SYN flood attack indicating that Ryu's event-driven architecture introduces a slight overhead when handling high volumes of attack traffic, affecting the performance of legitimate traffic more than POX.

6.6.3 Other factors

Expanding further and to add a nuanced comparison, the ease of integration (with scripts) is also a factor to consider. Running scripts side by side with the controllers can be key in solidifying the security of an SDN environment.

In this project, POX was the easiest to run scripts with. Its simplicity allowed it to achieve faster mitigation times across all experiments. It already had different modules that could be called like the forwarding switch module, and so we only had to add the IDPS and run it together. Ryu however, needed the IDPS to have forwarding switch logic alongside the script or else network flow was impossible. It's event-driven architecture offers flexibility and scalability no less, but its mitigation times lag behind POX in most scenarios except for the ICMP flood attack. While Ryu might be better suited for environments that require advanced customization and modularity, the slight trade-off in mitigation speed should be considered.

It is important for me to acknowledge that the scripts might not have been written in the best possible way to enable the controllers optimize for speed better than they did. This should be taken into account.

7 Conclusion

In this study, we implemented and evaluated an Intrusion Detection and Prevention System (IDPS) using SDN to mitigate three common types of DoS attacks: SYN Flood, Port Scanning, and ICMP Flood attacks. The controllers tested- POX, Ryu were selected based on their popularity, ease of use, and open-source availability, which are critical for practical deployment in SDN environments.

The IDPS functioned as expected and blocked dropped traffic from suspicious hosts before blocking them. The experiments conducted in a controlled virtualized environment provided valuable insights into the capabilities of both controllers in detecting and mitigating these attacks within acceptable time frames, although with notable differences in performance.

From the experiments, POX comes out as the faster SDN controller in terms of attack detection and mitigation and therefore best for environments where speed and simplicity are the top priorities. Ryu remains a viable option for larger, more complex networks that benefit from modularity and scalability, however, the slight trade-off in mitigation speed and the potential impact on legitimate traffic should be considered.

Nearly all objectives were met, the only one which couldn’t be met was carrying out the experiments on Faucet controller. This is due to time constraint and complexity associated with configuring the YAML file to work with my testbed; hence, the comparison was just between POX controller and Ryu Controller.

7.1 Limitations and Future work

While this study provides a comparison of the selected controllers from a security perspective, it is limited by the scope of the attacks tested and the testing environment. The virtual environment used does not have the adequate network devices and bandwidth to mimic a real corporate environment scenario.

Faucet controller was originally planned to be tested as well, however due to time constraint, it could not be implemented.

Future work should explore additional attack vectors, such as application-layer DDoS attacks, more controllers (developed in other languages like C and Java), and test them in a more diverse environment, including physical network setups, to validate the generalizability of the findings.

Machine learning techniques could also prove efficient in enhancing detection accuracy. It could be added as an extra layer in conjunction with traditional methods as this.

8 APPENDIX

APPENDIX A

OvS Installation

Run terminal commands as root

```
>sudo s

>apt-get install y git
```

Install OvS by cloning using git

```
>git clone https://github.com/openvswitch/OvS. Git
```

Change into OvS directory

```
>cd OvS
```

Before proceeding, install dependencies:

```
>apt-get install -y build-essential libssl-dev libcap-
ng-utils

>apt-get install -y python-setuptools

>apt-get install -y python-pyftpdlib wget netcat curl

>pip install pip

>apt install python3-pip

>apt install -y gambc
```

Run bootstrap and GNU make in the build directory

```
>./boot.sh

>  ./configure  --prefix=/usr  --localstatedir=/var  --
sysconfdir=/etc

>Make

>Make install
```

Load the kernel module and check that it is loaded

```
>/sbin/modprobe openvswitch

>/sbin/lsmod | grep openvswitch
```

Export path where OvS utility scripts are located

```
 >export
PATH=$PATH:/usr/local/share/openvswitch/scripts
```

OvS Configuration

Configure the OvSdb-server

```
 >mkdir -p /usr/local/etc/openvswitch

 >OvSdb-tool create /usr/local/etc/openvswitch/conf.db
vswitchd/vswitch.OvSschema

 >mkdir -p /usr/local/var/run/openvswitch
```

Start OvS, you should get a response telling you that it is up

```
 >/usr/share/openvswitch/scripts/OvS-ctl start
```

Create a bridge, called OvSbridge and bring interface up

```
 >OvS-vsctl add-br OvSbridge
 >Ifconfig OvSbridge up
```

Add virtual network adapters

```
 >ip tuntap add mode tap vport1
 >ip tuntap add mode tap vport2
 >ip tuntap add mode tap vport3
 >ip tuntap add mode tap vport4
```

Bring the links up

```
 >ip link set vport1 up
 >ip link set vport2 up
 >ip link set vport3 up
 >ip link set vport4 up
```

Add virtual adapters to the OvSbridge

```
 >OvS-vsctl add-port OvSbridge vport1
 >OvS-vsctl add-port OvSbridge vport2
 >OvS-vsctl add-port OvSbridge vport3
 >OvS-vsctl add-port OvSbridge vport4
```

Set the OvSbridge IP

```
 >ifconfig OvSbridge 10.0.2.10 netmask 255.255.255.0 up
```

Configure OvS to connect to a primary controller and set secure mode

```
 >OvS-vsctl set-controller OvSbridge tcp:10.0.2.11:6633
 >OvS-vsctl set-fail-mode OvSbridge secure
```

Finally show OvS details to see your config

```
>OvS-vsctl show
```

APPENDIX B

Code ran on the Traffic-VM to get HTTP requests from the Victim-VM. A way to measure RTT

```
 1. import requests
 2. import sched
 3. import time
 4.
 5. #Initialize the scheduler
 6. s= sched.scheduler(time.time, time.sleep)
 7.
 8. def test_server(sc):
 9.     try:
10.             #This is to send GET request and measure
elapsed time
11.             response_time =
requests.get("http://10.0.2.15").elapsed.total_second
s()
12.             timestamp = time.asctime()
13.
14.             #Print out the current time and response
time
15.             print(response_time, timestamp)
16.
17.             #schedule the next run of this function
18.             s.enter(1,1, test_server, (sc,))
19.     except requests.exceptions.RequestException as
e:
20.             print(f"Request failed: {e}, Time:
{time.aaaasctime()}")
21.             s.enter(1, 1, test_server, (sc,))
22.
23. #Schedule the first test_server run
24. s.enter(1, 1, test_server, (s,))
25.
26. #Start the scheduler
27. s.run()
28.
```

APPENDIX C

The codes in Appendix C, D, E were created and modified for this master's project. They can also be found in github-- https://github.com/mannylogic/SDN_IDPS/.

Python IDPS Code port scanning attack- POX controller

```
from pox.core import core
from pox.lib.revent import EventHalt
import pox.openflow.libopenflow_01 as of
from datetime import datetime
from time import time

# Create a logger for this component
log = core.getLogger()
log.info("IDPS going up %s", datetime.now())

threshold = 100  # Attack detection threshold
connections = {}  # Tracked connections
attacker = set()  # Blacklist

def tcp_func(event, c=None, t=threshold):
    """Receive PacketIn and process TCP packets."""
    if c is None:
        c = connections
    pkt = event.parsed

    mac_blocker(str(pkt.src))  # Kill blocked packets

    cname = str(pkt.src) + ' => ' + str(pkt.dst)
    p = event.parsed.find('tcp')  # TCP packet attributes

    if not p:
        return

    if True in [p.FIN, p.RST, p.PSH, p.URG, p.ECN, p.CWR]:
        return

    if p.SYN and not p.ACK:
        if cname not in c:
            new_connection(cname, pkt)
        elif check_timeout(cname):
            del c[cname]
            return

        trw(cname, syn=True, ack=False)
        rl(cname)

        if c[cname]['trw'] > t:
            detect_and_mitigate_attack('trw', cname)
        elif c[cname]['rl'] > t:
            detect_and_mitigate_attack('rl', cname)

    elif p.SYN and p.ACK:
```

```
48.         rev_cname = str(pkt.dst) + ' => ' + str(pkt.src)
49.         if rev_cname in c:
50.             trw(rev_cname, syn=True, ack=True)
51.
52. def mac_blocker(ethernet_src):
53.     """MAC blocker - kill packets from attackers."""
54.     global attacker
55.     if ethernet_src in attacker:
56.         return EventHalt
57.
58. def new_connection(cname, pkt, c=None):
59.     """Track new connections."""
60.     if c is None:
61.         c = connections
62.     c[cname] = {
63.         'src': pkt.src,
64.         'dst': pkt.dst,
65.         'trw': 0,
66.         'rl': 0,
67.         'time': time(),
68.         'detection_time': None,
69.         'mitigation_time': None
70.     }
71.
72. def check_timeout(cname, c=None):
73.     """Time-out tracked connections after a minute."""
74.     if c is None:
75.         c = connections
76.     if time() - c[cname]['time'] > 60:
77.         return True
78.     else:
79.         return False
80.
81. def trw(cname, syn, ack, c=None, t=threshold):
82.     """Implementation of CB-TRW algorithm."""
83.     if c is None:
84.         c = connections
85.     log.debug('trw %s' % str(c[cname]['trw']), syn, ack)
86.     if syn and not ack:
87.         c[cname]['trw'] += 1
88.     elif syn and ack:
89.         c[cname]['trw'] -= 1
90.
91. def rl(cname, c=None, t=threshold):
92.     """Implementation of Rate Limiting algorithm."""
93.     if c is None:
94.         c = connections
95.     log.debug('rl %s' % str(c[cname]['rl']))
96.     c[cname]['rl'] += 1
97.
98. def detect_and_mitigate_attack(algo, cname, c=None):
99.     """Detection and Mitigation Process."""
100.     if c is None:
101.         c = connections
102.     # Record the detection time
103.     if not c[cname]['detection_time']:
104.         c[cname]['detection_time'] = datetime.now()
105.         log.info('Attack detected by %s: %s at %s' % (algo,
cname, c[cname]['detection_time']))
106.
107.     # Perform mitigation
```

```
108.     c[cname]['mitigation_time'] = datetime.now()
109.     log.warning('Mitigation action taken by %s for %s at
%s' % (algo, cname, c[cname]['mitigation_time']))
110.
111.     # Add to attacker blacklist and send flow mod to block
112.     attacker.add(str(c[cname]['src']))
113.     msg = of.ofp_flow_mod()
114.     msg.priority = 65535
115.     msg.match.dl_src = c[cname]['src']
116.     for switch in core.openflow.connections:
117.         switch.send(msg)
118.
119.     # Calculate and log the time difference between
detection and mitigation
120.     detection_to_mitigation_time =
(c[cname]['mitigation_time'] -
c[cname]['detection_time']).total_seconds()
121.     log.info('Speed of mitigation for %s: %s seconds' %
(cname, detection_to_mitigation_time))
122.
123.     # Remove the connection record
124.     del c[cname]
125.
126. def clear_flows():
127.     """Clear flows on all switches."""
128.     delete_command =
of.ofp_flow_mod_command_rev_map['OFPFC_DELETE']
129.     d = of.ofp_flow_mod(command=delete_command)
130.     for switch in core.openflow.connections:
131.         switch.send(d)
132.
133. def launch():
134.     core.openflow.addListenerByName("PacketIn", tcp_func,
priority=1)
135.
```

Python IDPS Code for port scanning attack- Ryu controller

```
  1. from ryu.base import app_manager
  2. from datetime import datetime
  3. import time
  4. from ryu.controller.handler import set_ev_cls,
CONFIG_DISPATCHER, MAIN_DISPATCHER
  5. from ryu.controller import ofp_event
  6.
  7. from ryu.lib.packet import ethernet, ipv4, tcp, packet
  8.


  9.
 10.
 11. class EnhancedIDPS(app_manager.RyuApp):
 12.     OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]
 13.
 14.     def __init__(self, *args, **kwargs):
 15.         super(EnhancedIDPS, self).__init__(*args, **kwargs)
 16.         self.connections = {}
 17.         self.attacker = set()
 18.         self.threshold = 100  # Attack detection threshold
```

```
 19.         self.logger.info("Ryu IDPS starting up at %s",
datetime.now())
 20.
 21.     @set_ev_cls(ofp_event.EventOFPSwitchFeatures,
CONFIG_DISPATCHER)
 22.     def switch_features_handler(self, ev):
 23.         datapath = ev.msg.datapath
 24.         ofproto = datapath.ofproto
 25.         parser = datapath.ofproto_parser
 26.
 27.         # Install the table-miss flow entry to send unknown
packets to the controller
 28.         match = parser.OFPMatch()
 29.         actions =
[parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
ofproto.OFPCML_NO_BUFFER)]
 30.         self.install_flow(datapath, 0, match, actions)
 31.
 32.     def add_flow(self, datapath, priority, match, actions):
 33.         ofproto = datapath.ofproto
 34.         parser = datapath.ofproto_parser
 35.
 36.         instructions =
[parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
actions)]
 37.
 38.         mod = parser.OFPFlowMod(
 39.             datapath=datapath, priority=priority,
match=match,
 40.             instructions=inst)
 41.         datapath.send_msg(mod)
 42.
 43.     @set_ev_cls(ofp_event.EventOFPPacketIn,
MAIN_DISPATCHER)
 44.     def _handle_packet_in(self, ev):
 45.         msg = ev.msg
 46.         datapath = msg.datapath
 47.         ofproto = datapath.ofproto
 48.         parser = datapath.ofproto_parser
 49.         in_port = msg.match['in_port']
 50.
 51.         pkt = packet.Packet(msg.data)
 52.         eth = pkt.get_protocol(ethernet.ethernet)
 53.
 54.         # Ignore packets from attackers
 55.         if eth.src in self.attacker:
 56.             return
 57.
 58.         # Handle ARP packets
 59.         if eth.ethertype == 0x0806:
 60.             self.handle_arp(datapath, in_port, pkt)
 61.             return
 62.
 63.         ip_pkt = pkt.get_protocol(ipv4.ipv4)
 64.         tcp_pkt = pkt.get_protocol(tcp.tcp)
 65.
 66.         if not tcp_pkt or tcp_pkt.bits & (tcp.TCP_SYN |
tcp.TCP_ACK) != tcp.TCP_SYN:
 67.             # Forward other packets normally
 68.             self.forward_packet(datapath, in_port,
msg.data)
```

```
 69.             return
 70.
 71.         cname = str(ip_pkt.src) + ' => ' + str(ip_pkt.dst)
 72.
 73.         if cname not in self.connections:
 74.             self.new_connection(cname, ip_pkt.src,
ip_pkt.dst)
 75.         elif self.check_timeout(cname):
 76.             del self.connections[cname]
 77.             return
 78.
 79.         if cname in self.connections:
 80.             self.trw(cname, tcp_pkt)
 81.             self.rl(cname)
 82.
 83.             if self.connections[cname]['trw'] >
self.threshold:
 84.                 self.mitigate_attack(datapath, ip_pkt.src,
'trw', cname)
 85.             elif self.connections[cname]['rl'] >
self.threshold:
 86.                 self.mitigate_attack(datapath, ip_pkt.src,
'rl', cname)
 87.             else:
 88.                 # Forward packets of non-attack connections
 89.                 self.forward_packet(datapath, in_port,
msg.data)
 90.
 91.     def forward_packet(self, datapath, in_port, data):
 92.         """Forward packets to the appropriate port."""
 93.         ofproto = datapath.ofproto
 94.         parser = datapath.ofproto_parser
 95.         actions =
[parser.OFPActionOutput(ofproto.OFPP_FLOOD)]
 96.         out = parser.OFPPacketOut(
 97.             datapath=datapath,
buffer_id=ofproto.OFP_NO_BUFFER,
 98.             in_port=in_port, actions=actions, data=data)
 99.         datapath.send_msg(out)
100.
101.     def handle_arp(self, datapath, in_port, pkt):
102.         """Handle ARP requests and replies."""
103.         ofproto = datapath.ofproto
104.         parser = datapath.ofproto_parser
105.
106.         eth_pkt = pkt.get_protocol(ethernet.ethernet)
107.         arp_pkt = pkt.get_protocol(arp.arp)
108.
109.         # Create ARP reply or forward ARP request
110.         if arp_pkt.opcode == arp.ARP_REQUEST:
111.             self.forward_packet(datapath, in_port,
pkt.data)
112.         elif arp_pkt.opcode == arp.ARP_REPLY:
113.             self.forward_packet(datapath, in_port,
pkt.data)
114.
115.     def new_connection(self, cname, src_ip, dst_ip):
116.         self.connections[cname] = {
117.             'src': src_ip,
118.             'dst': dst_ip,
119.             'trw': 0,
```

```
120.                 'rl': 0,
121.                 'start_time': time.time()
122.             }
123.
124.     def check_timeout(self, cname):
125.         return time.time() -
self.connections[cname]['start_time'] > 60
126.
127.     def trw(self, cname, tcp_pkt):
128.         if tcp_pkt.bits == tcp.TCP_SYN:
129.             self.connections[cname]['trw'] += 1
130.         elif tcp_pkt.bits == (tcp.TCP_SYN | tcp.TCP_ACK):
131.             self.connections[cname]['trw'] -= 1
132.
133.     def rl(self, cname):
134.         self.connections[cname]['rl'] += 1
135.
136.     def mitigate_attack(self, datapath, src_ip, algo,
cname):
137.         if src_ip in self.attacker:
138.             return
139.
140.         detection_time = time.time()
141.         self.logger.info('Attack detected: %s at %s',
cname, datetime.now().time())
142.         self.attacker.add(src_ip)
143.
144.         # Use ipv4_src instead of eth_src for IP addresses
145.         match =
datapath.ofproto_parser.OFPMatch(ipv4_src=src_ip)
146.         actions = []
147.         self.install_flow(datapath, 65535, match, actions)
148.
149.         mitigation_time = time.time()
150.         self.logger.warning('Mitigation action taken for %s
at %s', cname, datetime.now().time())
151.
152.         speed_of_mitigation = mitigation_time -
self.connections[cname]['start_time']
153.         self.logger.info('Speed of mitigation: %.6f
seconds', speed_of_mitigation)
154.
155.         del self.connections[cname]
156.
```

APPENDIX D

Python IDPS code for SYN flood attack- POX controller

```
from pox.core import core
from pox.lib.revent import EventHalt
import pox.openflow.libopenflow_01 as of
from pox.lib.packet.ethernet import ethernet
from pox.lib.packet.ipv4 import ipv4
from pox.lib.packet.tcp import tcp
from datetime import datetime
from time import time

log = core.getLogger()
log.info("TCP SYN Flood IDPS with MAC blocking going up %s", datetime.now())

# Parameters
threshold = 50  # SYN packet rate threshold
connections = {}  # Tracked connections
attacker = set()  # Blacklist for MAC addresses

def tcp_func(event):
    """Receive PacketIn and process TCP packets."""
    pkt = event.parsed

    eth = pkt.find('ethernet')
    ipv4_pkt = pkt.find('ipv4')
    tcp_pkt = pkt.find('tcp')

    if not eth or not ipv4_pkt or not tcp_pkt:
        return

    mac_src = eth.src

    # Block traffic from known attackers (based on MAC address)
    if mac_src in attacker:
        return EventHalt

    if tcp_pkt.SYN and not tcp_pkt.ACK:  # SYN packet
        cname = f"{mac_src} => {eth.dst}"

        if cname not in connections:
            new_connection(cname, pkt)
        elif check_timeout(cname):
            del connections[cname]
            return

        rl(cname)  # Apply rate limiting for SYN packets

        if connections[cname]['rl'] > threshold:
            detect_and_mitigate_attack('rl', cname)

def new_connection(cname, pkt):
    """Track new TCP connections based on MAC address."""
    eth = pkt.find('ethernet')
    connections[cname] = {
        'src': eth.src,
        'dst': eth.dst,
```

```
 55.         'rl': 0,
 56.         'time': time(),
 57.         'detection_time': None,
 58.         'mitigation_time': None
 59.     }
 60.
 61. def check_timeout(cname):
 62.     """Timeout tracked connections after a minute."""
 63.     return time() - connections[cname]['time'] > 60
 64.
 65. def rl(cname):
 66.     """Rate Limiting based on SYN packet rate."""
 67.     connections[cname]['rl'] += 1
 68.
 69. def detect_and_mitigate_attack(algo, cname):
 70.     """Detection and Mitigation Process."""
 71.     if not connections[cname]['detection_time']:
 72.         connections[cname]['detection_time'] =
datetime.now()
 73.         log.info(f"Attack detected by {algo}: {cname} at
{connections[cname]['detection_time']}")
 74.
 75.     # Mitigation
 76.     connections[cname]['mitigation_time'] = datetime.now()
 77.     log.warning(f"Mitigation action taken by {algo} for
{cname} at {connections[cname]['mitigation_time']}")
 78.     mac_src = connections[cname]['src']
 79.     attacker.add(mac_src)
 80.
 81.     # Send flow mod to block source MAC address
 82.     msg = of.ofp_flow_mod()
 83.     msg.priority = 65535
 84.     msg.match.dl_src = mac_src  # Block based on MAC
address
 85.
msg.actions.append(of.ofp_action_output(port=of.OFPP_NONE))  #
Drop the packet
 86.     for switch in core.openflow.connections:
 87.         switch.send(msg)
 88.
 89.     detection_to_mitigation_time =
(connections[cname]['mitigation_time'] -
connections[cname]['detection_time']).total_seconds()
 90.     log.info(f"Speed of mitigation for {cname}:
{detection_to_mitigation_time} seconds")
 91.
 92.     del connections[cname]
 93.
 94. def clear_flows():
 95.     """Clear flows on all switches."""
 96.     delete_command =
of.ofp_flow_mod_command_rev_map['OFPFC_DELETE']
 97.     d = of.ofp_flow_mod(command=delete_command)
 98.     for switch in core.openflow.connections:
 99.         switch.send(d)
100.
101. def launch():
102.     core.openflow.addListenerByName("PacketIn", tcp_func,
priority=1)
```

```
        log.info("TCP SYN Flood IDPS with MAC blocking
launched")
```

Python IDPS code for SYN flood attack- Ryu controller

```
  1. ryu.controller import ofp_event
  2. from ryu.base import app_manager
  3. from ryu.controller.handler import CONFIG_DISPATCHER,
MAIN_DISPATCHER, import set_ev_cls
  4. from ryu.ofproto import ofproto_v1_3
  5.
  6.
  7. from ryu.lib.packet import ethernet, ipv4, tcp, packet
  8. from datetime import datetime
  9. import time
 10.
 11. class SynFloodMitigation(app_manager.RyuApp):
 12.     OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]
 13.
 14.     def __init__(self, *args, **kwargs):
 15.         super(SynFloodMitigation, self).__init__(*args,
**kwargs)
 16.         self.connections = {}
 17.         self.attacker = set()
 18.         self.threshold = 50  # Attack detection threshold
 19.         self.logger.info("SYN Flood IDPS starting up at
%s", datetime.now())
 20.
 21.     @set_ev_cls(ofp_event.EventOFPSwitchFeatures,
CONFIG_DISPATCHER)
 22.     def switch_features_handler(self, ev):
 23.         datapath = ev.msg.datapath
 24.         ofproto = datapath.ofproto
 25.         parser = datapath.ofproto_parser
 26.
 27.         # Install the table-miss flow entry to send unknown
packets to the controller
 28.         match = parser.OFPMatch()
 29.         actions =
[parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
ofproto.OFPCML_NO_BUFFER)]
 30.         self.install_flow(datapath, 0, match, actions)
 31.
 32.     def add_flow(self, datapath, priority, match, actions):
 33.         ofproto = datapath.ofproto
 34.         parser = datapath.ofproto_parser
 35.
 36.         instructions =
[parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
actions)]
 37.
 38.         mod = parser.OFPFlowMod(
 39.             datapath=datapath, priority=priority,
match=match,
 40.             instructions=inst)
 41.         datapath.send_msg(mod)
 42.
 43.     @set_ev_cls(ofp_event.EventOFPPacketIn,
MAIN_DISPATCHER)
```

```
 44.     def _handle_packet_in(self, ev):
 45.         msg = ev.msg
 46.         datapath = msg.datapath
 47.         ofproto = datapath.ofproto
 48.         parser = datapath.ofproto_parser
 49.         in_port = msg.match['in_port']
 50.
 51.         pkt = packet.Packet(msg.data)
 52.         eth = pkt.get_protocol(ethernet.ethernet)
 53.
 54.         # Ignore packets from attackers
 55.         if eth.src in self.attacker:
 56.             return
 57.
 58.         ip_pkt = pkt.get_protocol(ipv4.ipv4)
 59.         tcp_pkt = pkt.get_protocol(tcp.tcp)
 60.
 61.         if not tcp_pkt or tcp_pkt.bits & (tcp.TCP_SYN |
tcp.TCP_ACK) != tcp.TCP_SYN:
 62.             # Forward other packets normally
 63.             self.forward_packet(datapath, in_port,
msg.data)
 64.             return
 65.
 66.         cname = f"{ip_pkt.src} => {ip_pkt.dst}"
 67.
 68.         if cname not in self.connections:
 69.             self.new_connection(cname, eth.src, eth.dst)
 70.         elif self.check_timeout(cname):
 71.             del self.connections[cname]
 72.             return
 73.
 74.         if cname in self.connections:
 75.             self.trw(cname, tcp_pkt)
 76.             self.rl(cname)
 77.
 78.             if self.connections[cname]['trw'] >
self.threshold:
 79.                 self.mitigate_attack(datapath, eth.src,
'trw', cname)
 80.             elif self.connections[cname]['rl'] >
self.threshold:
 81.                 self.mitigate_attack(datapath, eth.src,
'rl', cname)
 82.             else:
 83.                 # Forward packets of non-attack connections
 84.                 self.forward_packet(datapath, in_port,
msg.data)
 85.
 86.     def forward_packet(self, datapath, in_port, data):
 87.         """Forward packets to the appropriate port."""
 88.         ofproto = datapath.ofproto
 89.         parser = datapath.ofproto_parser
 90.         actions =
[parser.OFPActionOutput(ofproto.OFPP_FLOOD)]
 91.         out = parser.OFPPacketOut(
 92.             datapath=datapath,
buffer_id=ofproto.OFP_NO_BUFFER,
 93.             in_port=in_port, actions=actions, data=data)
 94.         datapath.send_msg(out)
 95.
```

```
 96.     def new_connection(self, cname, src_mac, dst_mac):
 97.         self.connections[cname] = {
 98.             'src_mac': src_mac,
 99.             'dst_mac': dst_mac,
100.             'trw': 0,
101.             'rl': 0,
102.             'start_time': time.time()
103.         }
104.
105.     def check_timeout(self, cname):
106.         return time.time() -
self.connections[cname]['start_time'] > 60
107.
108.     def trw(self, cname, tcp_pkt):
109.         if tcp_pkt.bits == tcp.TCP_SYN:
110.             self.connections[cname]['trw'] += 1
111.         elif tcp_pkt.bits == (tcp.TCP_SYN | tcp.TCP_ACK):
112.             self.connections[cname]['trw'] -= 1
113.
114.     def rl(self, cname):
115.         self.connections[cname]['rl'] += 1
116.
117.     def mitigate_attack(self, datapath, src_mac, algo,
cname):
118.         if src_mac in self.attacker:
119.             return
120.
121.         detection_time = time.time()
122.         self.logger.info('Attack detected by %s: %s at %s',
algo, cname, datetime.now().time())
123.         self.attacker.add(src_mac)
124.
125.         match =
datapath.ofproto_parser.OFPMatch(eth_src=src_mac)
126.         actions = []
127.         self.install_flow(datapath, 65535, match, actions)
128.
129.         mitigation_time = time.time()
130.         self.logger.warning('Mitigation action taken by %s
for %s at %s', algo, cname, datetime.now().time())
131.
132.         # Calculate the speed of mitigation correctly
133.         speed_of_mitigation = mitigation_time -
detection_time
134.         self.logger.info('Speed of mitigation: %.6f
seconds', speed_of_mitigation)
135.
136.         del self.connections[cname]
137.
```

APPENDIX E

Python IDPS code for ICMP flood attack- POX controller

```
from pox.core import core
from pox.lib.revent import EventHalt
import pox.openflow.libopenflow_01 as of
from pox.lib.packet.ethernet import ethernet
from pox.lib.packet.ipv4 import ipv4
from pox.lib.packet.icmp import icmp
from datetime import datetime
from time import time

log = core.getLogger()
log.info("ICMP Flood IDPS with MAC blocking going up %s",
datetime.now())

# Parameters
threshold = 50  # ICMP packet rate threshold
connections = {}  # Tracked connections
attacker = set()  # Blacklist for MAC addresses

def icmp_func(event):
    """Receive PacketIn and process ICMP packets."""
    pkt = event.parsed

    eth = pkt.find('ethernet')
    icmp_pkt = pkt.find('icmp')

    if not eth or not icmp_pkt:
        return

    mac_src = eth.src

    # Block traffic from known attackers (based on MAC
address)
    if mac_src in attacker:
        return EventHalt

    cname = f"{mac_src} => {eth.dst}"

    if cname not in connections:
        new_connection(cname, pkt)
    elif check_timeout(cname):
        del connections[cname]
        return

    rl(cname)  # Apply rate limiting for ICMP packets

    if connections[cname]['rl'] > threshold:
        detect_and_mitigate_attack('rl', cname)

def new_connection(cname, pkt):
    """Track new ICMP connections based on MAC address."""
    eth = pkt.find('ethernet')
    connections[cname] = {
        'src': eth.src,
        'dst': eth.dst,
        'rl': 0,
        'time': time(),
```

```
 55.         'detection_time': None,
 56.         'mitigation_time': None
 57.     }
 58.
 59. def check_timeout(cname):
 60.     """Timeout tracked connections after a minute."""
 61.     return time() - connections[cname]['time'] > 60
 62.
 63. def rl(cname):
 64.     """Rate Limiting based on ICMP packet rate."""
 65.     connections[cname]['rl'] += 1
 66.
 67. def detect_and_mitigate_attack(algo, cname):
 68.     """Detection and Mitigation Process."""
 69.     if not connections[cname]['detection_time']:
 70.         connections[cname]['detection_time'] =
datetime.now()
 71.         log.info(f"ICMP Flood attack detected by {algo}:
{cname} at {connections[cname]['detection_time']}")
 72.
 73.     # Mitigation
 74.     connections[cname]['mitigation_time'] = datetime.now()
 75.     log.warning(f"Mitigation action taken by {algo} for
{cname} at {connections[cname]['mitigation_time']}")
 76.     mac_src = connections[cname]['src']
 77.     attacker.add(mac_src)
 78.
 79.     # Send flow mod to block source MAC address
 80.     msg = of.ofp_flow_mod()
 81.     msg.priority = 65535
 82.     msg.match.dl_src = mac_src  # Block based on MAC
address
 83.
msg.actions.append(of.ofp_action_output(port=of.OFPP_NONE))  #
Drop the packet
 84.     for switch in core.openflow.connections:
 85.         switch.send(msg)
 86.
 87.     detection_to_mitigation_time =
(connections[cname]['mitigation_time'] -
connections[cname]['detection_time']).total_seconds()
 88.     log.info(f"Speed of mitigation for {cname}:
{detection_to_mitigation_time} seconds")
 89.
 90.     del connections[cname]
 91.
 92. def clear_flows():
 93.     """Clear flows on all switches."""
 94.     delete_command =
of.ofp_flow_mod_command_rev_map['OFPFC_DELETE']
 95.     d = of.ofp_flow_mod(command=delete_command)
 96.     for switch in core.openflow.connections:
 97.         switch.send(d)
 98.
 99. def launch():
100.     core.openflow.addListenerByName("PacketIn", icmp_func,
priority=1)
101.     log.info("ICMP Flood IDPS with MAC blocking launched")
102.
```

Python IDPS code for ICMP flood attack- Ryu controller

```
  1. from ryu.base import app_manager
  2. from ryu.controller import ofp_event
  3. from ryu.controller.handler import CONFIG_DISPATCHER,
MAIN_DISPATCHER, import set_ev_cls
  4.
  5. from ryu.ofproto import ofproto_v1_3
  6. from ryu.lib.packet import packet
  7. from ryu.lib.packet import ethernet, icmp, ipv4, import
ether_types
  8.
  9. from datetime import datetime
 10. from time import time
 11.
 12. # Define global variables
 13. threshold = 50  # Attack detection threshold
 14. connections = {}  # Tracked connections
 15. attacker = set()  # Blacklist
 16.
 17. class SimpleSwitchIDPS(app_manager.RyuApp):
 18.     OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]
 19.
 20.     def __init__(self, *args, **kwargs):
 21.         super(SimpleSwitchIDPS, self).__init__(*args,
**kwargs)
 22.         self.mac_to_port = {}
 23.         self.logger.info("ICMP Flood IDPS with Simple
Switch functionality started at %s", datetime.now())
 24.
 25.     @set_ev_cls(ofp_event.EventOFPSwitchFeatures,
CONFIG_DISPATCHER)
 26.     def switch_features_handler(self, ev):
 27.         datapath = ev.msg.datapath
 28.         ofproto = datapath.ofproto
 29.         parser = datapath.ofproto_parser
 30.
 31.         match = parser.OFPMatch()
 32.         actions =
[parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
ofproto.OFPCML_NO_BUFFER)]
 33.         self.install_flow(datapath, 0, match, actions)
 34.
 35.     def add_flow(self, datapath, priority, match, actions,
buffer_id=None):
 36.         ofproto = datapath.ofproto
 37.         parser = datapath.ofproto_parser
 38.
 39.         instructions =
[parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
actions)]
 40.         if buffer_id:
 41.             mod = parser.OFPFlowMod(datapath=datapath,
buffer_id=buffer_id, priority=priority,
 42.                                     match=match,
instructions=inst)
 43.         else:
 44.             mod = parser.OFPFlowMod(datapath=datapath,
priority=priority,
```

```
 45.                                     match=match,
instructions=inst)
 46.         datapath.send_msg(mod)
 47.
 48.     @set_ev_cls(ofp_event.EventOFPPacketIn,
MAIN_DISPATCHER)
 49.     def _handle_packet_in(self, ev):
 50.         if ev.msg.msg_len < ev.msg.total_len:
 51.             self.logger.debug("Packet truncated: only %s of
%s bytes", ev.msg.msg_len, ev.msg.total_len)
 52.
 53.         msg = ev.msg
 54.         datapath = msg.datapath
 55.         ofproto = datapath.ofproto
 56.         parser = datapath.ofproto_parser
 57.         in_port = msg.match['in_port']
 58.
 59.         pkt = packet.Packet(msg.data)
 60.         eth = pkt.get_protocols(ethernet.ethernet)[0]
 61.
 62.         dst = eth.dst
 63.         src = eth.src
 64.         dpid = datapath.id
 65.         self.mac_to_port.setdefault(dpid, {})
 66.
 67.         # Learn a MAC address to avoid FLOOD next time.
 68.         self.mac_to_port[dpid][src] = in_port
 69.
 70.         # ICMP handling and IDPS
 71.         if eth.ethertype == ether_types.ETH_TYPE_IP:
 72.             ip_pkt = pkt.get_protocol(ipv4.ipv4)
 73.             if ip_pkt.proto == ipv4.inet.IPPROTO_ICMP:
 74.                 self.icmp_func(src, dst, datapath, in_port)
 75.
 76.         # Simple switch functionality
 77.         if dst in self.mac_to_port[dpid]:
 78.             out_port = self.mac_to_port[dpid][dst]
 79.         else:
 80.             out_port = ofproto.OFPP_FLOOD
 81.
 82.         actions = [parser.OFPActionOutput(out_port)]
 83.
 84.         # Install a flow to avoid packet_in next time
 85.         if out_port != ofproto.OFPP_FLOOD:
 86.             match = parser.OFPMatch(in_port=in_port,
eth_dst=dst)
 87.             if msg.buffer_id != ofproto.OFP_NO_BUFFER:
 88.                 self.install_flow(datapath, 1, match,
actions, msg.buffer_id)
 89.                 return
 90.             else:
 91.                 self.install_flow(datapath, 1, match,
actions)
 92.
 93.         data = None
 94.         if msg.buffer_id == ofproto.OFP_NO_BUFFER:
 95.             data = msg.data
 96.
 97.         out = parser.OFPPacketOut(datapath=datapath,
buffer_id=msg.buffer_id, in_port=in_port,
```

```
 98.                                     actions=actions,
data=data)
 99.         datapath.send_msg(out)
100.
101.     def mac_blocker(self, src, datapath):
102.         """MAC blocker - kill packets from attackers."""
103.         if src in attacker:
104.             self.logger.info("Blocking traffic from
attacker MAC: %s", src)
105.             match =
datapath.ofproto_parser.OFPMatch(eth_src=src)
106.             mod =
datapath.ofproto_parser.OFPFlowMod(datapath=datapath,
priority=65535,
107.
match=match, instructions=[])
108.             datapath.send_msg(mod)
109.             return True
110.         return False
111.
112.     def icmp_func(self, src, dst, datapath, in_port):
113.         """Process ICMP packets and detect potential
floods."""
114.         global connections, attacker
115.
116.         if self.mac_blocker(src, datapath):  # Kill blocked
packets if necessary
117.             return
118.
119.         cname = src + ' => ' + dst
120.
121.         if cname not in connections:
122.             self.new_connection(cname, src, dst)
123.         elif self.check_timeout(cname):
124.             del connections[cname]
125.             return
126.
127.         self.rl(cname)  # Apply rate limiting for ICMP
packets
128.
129.         if connections[cname]['rl'] > threshold:
130.             self.detect_and_mitigate_attack('rl', cname,
datapath)
131.
132.     def new_connection(self, cname, src, dst):
133.         """Track new ICMP connections."""
134.         global connections
135.         connections[cname] = {
136.             'src': src,
137.             'dst': dst,
138.             'rl': 0,
139.             'time': time(),
140.             'detection_time': None,
141.             'mitigation_time': None
142.         }
143.
144.     def check_timeout(self, cname):
145.         """Time-out tracked connections after a minute."""
146.         global connections
147.         if time() - connections[cname]['time'] > 60:
148.             return True
```

```
149.             return False
150.
151.         def rl(self, cname):
152.             """Implementation of Rate Limiting algorithm for
ICMP."""
153.             global connections
154.             connections[cname]['rl'] += 1
155.
156.         def detect_and_mitigate_attack(self, algo, cname,
datapath):
157.             """Detection and Mitigation Process."""
158.             global connections, attacker
159.
160.             if not connections[cname]['detection_time']:
161.                 connections[cname]['detection_time'] =
datetime.now()
162.                 self.logger.info('ICMP Flood Attack detected by
%s: %s at %s' % (algo, cname,
connections[cname]['detection_time']))
163.
164.             connections[cname]['mitigation_time'] =
datetime.now()
165.             self.logger.warning('Mitigation action taken by %s
for %s at %s' % (algo, cname,
connections[cname]['mitigation_time']))
166.
167.             attacker.add(connections[cname]['src'])
168.
169.             match =
datapath.ofproto_parser.OFPMatch(eth_src=connections[cname]['src
'])
170.             mod =
datapath.ofproto_parser.OFPFlowMod(datapath=datapath,
priority=65535,
171.
match=match, instructions=[])
172.             datapath.send_msg(mod)
173.
174.             detection_to_mitigation_time =
(connections[cname]['mitigation_time'] -
connections[cname]['detection_time']).total_seconds()
175.             self.logger.info('Speed of mitigation for %s: %s
seconds' % (cname, detection_to_mitigation_time))
176.
177.             del connections[cname]
178.
```

Bibliography

[1] Y. Chen, W. Sun, N. Zhang, Q. Zheng, W. Lou and T. Y. Hou, “Towards Efficient Fine-Grained Access Control and Trustworthy Data Processing for Remote Monitoring Services in IoT,” *IEEE Transactions on Information Forensics and Security,* vol. 14, pp. 1830-1842, 2019.

[2] B. Benamar, K. Benamar, H. Fouzi and S. Ying, “DDOS-attacks detection using an efficient measurement-based statistical mechanism,” *Engineering Science and Technology, an International Journal,* vol. 23, pp. 870-878, 2020.

[3] D. Kreutz, F. M. Ramos, P. E. Verissimo, C. E. Rothenberg, S. Azodolmolky and S. Uhlig, “Software-Defined Networking: A Comprehensive Survey,” *Proceedings of the IEEE,* vol. 103, no. 1, pp. 14-76, 2015.

[4] L. Vailshery, “Software-defined networking market revenue worldwide 2021-2028,” Statista, 2024.

[5] D. Dmitry, K. Eric and J. Rexford, “Scalable Network Virtualization in Software-Defined Networks,” *IEEE Internet Computing,* vol. 17, no. 2, pp. 20-27, 2012.

[6] S. Ola, E. Imad, K. Ayman and C. Ali, “SDN Controllers: A comparative study,” in *18th Mediterranean Electrotechnical Conference (MELECON)*, 2016.

[7] A. E. Fidha and A.-S. B. Mohammed, “Performance Analysis of Various SDN Controllers with Different Network Size in SDWN,” in *2nd International Multi-Disciplinary Conference Theme: Integrated Sciences and Technologies*, Sarkaya, 2022.

[8] A. Jellalah, A. Abdalwart, A. Abubaker, E. Abdulwahed, E. Wisam and S. O. Salem, “SDN Controllers Comparison Based On Network Topology,” in *2022 Workshop on Microwave Theory and Techniques in Wireless Communications (MTTW)*, Riga, 2022.

[9] O. Yoachimik and V. Ganti, “DDoS Attack Trends for Q4 2021,” Cloudflare, 10 January 2022. [Online]. Available: https://blog.cloudflare.com/ddos-attack-trends-for-2021-q4/.

[10] B. Celyn, R. Elpida and V. Vassilios, “Implementing an intrusion detection and prevention system using software-defined networking: : Defending against port-scanning and

denial-of-service attacks," *Journal of Network and Computer Applications,* vol. 136, pp. 71-85, 2019.

[11] Y. Lily, A. A. Todd, G. Ram and D. Ram, "Forwarding and Control Element Separation (ForCES) Framework," RFC Editor, 2004.

[12] M. Nick, A. Tom, B. Hari, P. Guru, P. Larry, R. Jennifer, S. Scott and T. Jonathan, "OpenFlow: Enabling Innovation in Campus Networks," *SIGCOMM Computer Communication,* vol. 38, no. 2, pp. 69-74, 2008.

[13] F. Nick and B. Hari, "'Detecting BGP configuration faults," in *2nd Conference on Symposium on Networked Systems Designs & Implementation*, 2005.

[14] Open Network Foundation, "Software Defined Networking," [Online]. Available: https://opennetworking.org/sdn-definition/. [Accessed July 2024].

[15] A. Bruno, C. Marc, N. Xuan, O. Katia and T. Thierry, "A Survey of Software-Defined Networking: Past, Present, and Future of Programmable Networks," *IEEE Communications Surveys & Tutorials,* vol. 5, 2014.

[16] Open Networking Foundation, "OpenFlow Switch Specification," 2013.

[17] N. Saritakumar, V. Adarsh, E. Srinavasan, S. Albert and R. Subha, "Performance Evaluation of Pox Controller for Software Defined Networks," *International Journal of Innovative Technology and Exploring Engineering (IJITEE),* vol. 8, no. 9, 2019.

[18] Open Networking Foundation, "Open Flow Switch Specification," 26 March 2015. [Online]. Available: https://www.opennetworking.org/wp-content/uploads/2014/10/openflow-switch-v1.5.1.pdf.

[19] RYU-SDN Framework Community, "Build SDN Agilely," 2017. [Online]. Available: ryu-sdn.org.

[20] M. Fabian, M. Mario, C. Symeon, L. Lei and V. Thang, "An SDN Based Testbed for Dynamic Network Slicing in Satellite-Terrestrial Networks," in *IEEE International Mediterranean*, Athens, Greece, 2021.

[21] R. Anil, P. D. Manash and K. C. Swarnendu, “A Flow-Based Performance Evaluation on RYU SDN Controller,” *Journal of The Institution of Engineers,* vol. 105, pp. 203-215, 2024.

[22] C. Ontiveros, “A SOFTWARE DEFINED NETWORK IMPLEMENTATION USING MININET AND RYU,” California State University Dominguez Hills , 2019.

[23] K. Vipin, S. Jaideep and L. Aleksandar, “Managing Cyber Threats,” in *Massive Computing*, 2005.

[24] H.-J. Liao, C.-H. R. Lin, Y.-C. Lin and K.-Y. Tung, “Intrusion detection system: A comprehensive review,” *Journal of Network and Computer Applications,* vol. 36, no. 1, pp. 16-24, 2013.

[25] D. Yuri and O. Erdal, Cybersecurity- Attack and Defense Strategies, Birmingham-Mumbai: Packt, 2018.

[26] P. Priteshkumar, B. Bhumika, Z. Gautam , A. Madhav and S. Parth , “A Review on Recent Intrusion Detection Systems and Intrusion Prevention Systems in IoT,” *International Conference on Cloud Computing, Data Science & Engineering,* vol. 11, 2021.

[27] National Cyber Security Centre, “Denial of Service (Dos) Guidance,” NCSC.gov.uk, 16 March 2016. [Online]. Available: https://www.ncsc.gov.uk/collection/denial-service-dos-guidance-collection.

[28] E. F. Lubna and P. Roberto, “DoS and DDoS attacks in Software Defined Networks: A survey of existing solutinos and research challenges,” *International Journal of eScience,* pp. 149-171, 2021.

[29] C. B. Lee, C. Roedel and E. Silenok, “Detection and Characterization of Port Scan Attacks,” Department of Computer Science & Engineering, San Diego.

[30] K. Geetha and N. Sreenath, “SYN flooding attack-Identification and analysis,” in *International Conference on Information Communication and Embedded Systems*, India, 2014.

[31] N. Gupta, A. Jain, P. Saini and V. Gupta, “DDoS attack algorithm using ICMP flood,” in *International Conference on Computing for Sustainable Global Development*, New Delhi, 2016.

[32] Oracle, “Virtual Box,” 2024. [Online]. Available: https://www.virtualbox.org/.

[33] Wireshark foundation, “About wireshark,” [Online]. Available: https://www.wireshark.org/about.html.

[34] G. Neelam, M. Mashael, T. Saarvesh, B. Sumit, A. Mohammed and B. Salil, “A Comparative Study of Software Defined Networking Controllers Using Mininet,” *Intelligent Data Sensing, Processing, Mining, and Communication,* vol. 11, no. 17, 2022.

[35] M. Rezi, J. Reza and C. Mauro, “SLICOTS: An SDN-based lightweight countermeasure for TCP SYN Flooding attacks,” *IEEE Transanctions on Network and Service Management,* vol. 14, no. 2, pp. 487-497, 2017.

[36] C. Tommy, M. Xenia, L. Xiangyang and X. Kaiqi, “Selective packet inspection to detect DoS flooding using software defined networking (SDN),” in *IEEE 35th International conference*, 2015.

[37] W. Wang, B. Yang and V. Y. Chen, “A visual analytics approach to detecting server redirections and data exfiltration,” in *IEEE International Conference on Intelligence and Security Informatics*, Baltimore, 2015.

[38] A. A and P. B, “Machine learning based intrusion detection system for software defined networks,” in *International Conference on Emerging Security Technologies*, 2017.

[39] C. Neu, C. Tatsch, R. Lunardi, R. Michelin, A. Orozco and A. Zorzo, “Lightweight IPS for port scan in OpenFlow SDN networks,” in *IEEE/IFIP Network Operations and Management Symposium*, Taipei, 2018.

[40] O. Daichi, G. Luis, I. Satoru, A. Toru and S. Takuo , “A proposal of port scan detection method based on Packet-In Messages in OpenFlow networks and its evaluation,” *International Journal of Network Management,* vol. 31, no. 6, 2021.

[41] D. Sergei, V. Andrei and L. Ivan, “A fuzzy logic-based information security management for software-defined networks,” in *International Conference on Advanced Communication Technology*, 2014.

[42] M. Seyed Mohammad and M. St-Hilaire, “Early detection of DDoS attacks against SDN Controllers,” *International*

Conference on Computing, Networking and Communications, pp. 77-81, 2015.

[43] T. Shuen-Chih, I. Liu, C.-T. Lu, C.-H. Chang and J.-S. Li, “Defending Cloud Computing Environment Against the Challenge of DDoS Attacks Based on Software Defined Network,” in *Advances in Intelligent Information Hiding and Multimedia Signal Processing*, 2016.

[44] B. Julien, N. Pierre-Alexis, R. Filippo, B. Matthieu and C. Vania, “Stateful monitoring for DDoS protection in Software defined networks,” in *IEEE Conference on Netowrk Softwarization*, 2017.

[45] C. Yunhe, Q. Qing, G. Chun, S. Guowei, T. Youliang, X. Huanlai and Y. Lianshan, “Towards DDoS detection mechanisms in Software-Defined Networking,” *Journal of Network and Computer Applications,* vol. 190, 2021.

[46] G. F and B. A, “Neural Network based protection of software defined network controller against distributed denial of service attacks,” *International Journal of Engineering, Transactions B: Applications,* vol. 30, no. 11, pp. 1714-1722, 2017.

[47] S. Reneilson, S. Danilo, S. Walter, R. Admilson and M. Edward, “Machine learning algorithms to detect DDoS attacks in SDN,” in *Concurrency and Computation: Practice and Experience*, 2020.

[48] B. Celyn, R. Elpida and V. Vassilios, “Implementing an intrusion detection and prevention system using software-defined networking: Defending against port-scanning and denial-of-service attacks,” *Journal of Network and Computer Applications,* vol. 136, pp. 71-85, 2019.

[49] K. Rajat and A. Markku, “‘Denial-of-service attacks in OpenFlow SDN Networks,” in *IFIP/IEEE Symposium on Integrated Network Management*, 2015.

[50] M. A. Syed, K. Junaid and K. A. Syed, “Revisiting TRaffic Anomaly Detection Using Software Defined Networking,” in *Recent Advances in Intrusion Detection - 14th International Symposium*, CA, 2011.

[51] G. Canepa, “10 Most Used Linux Distributions of All Time,” tecmint, 7 March 2024. [Online]. Available: https://www.tecmint.com/linux-distributions/.

[52] S. TV, “How do I install an old kernel?,” Fedora Project Organization, 20 November 2015. [Online]. Available: https://askubuntu.com/questions/700214/how-do-i-install-an-old-kernel.

[53] Open vSwitch, “Open vSwitch on Linux, FreeBSD and NetBSD,” 2016. [Online]. Available: https://docs.openvswitch.org/en/latest/intro/install/general/.